



UNIVERSIDAD
NACIONAL DEL OESTE



COMUNICACIÓN Y

REDES

TRABAJO PRÁCTICO

GRUPO 5

| Abril Ruiz Blagine | | Matias Daniel Correa |

| Gonzalo Leonel Farias | | Leon Lucas Della Paolera | | Jonathan Facundo Mamani |



CÁTEDRA

COMUNICACIÓN Y REDES



CONTENIDO

INFORME.....	3
PUNTO 2 – PING PONG	3
PUNTO 3 – ICMP Y ARP.....	5
CONCLUSIONES.....	27
ANEXO	¡Error! Marcador no definido.



Escuela de Informática

Licenciatura en Informática

Comunicación y Redes II

Trabajo Práctico de Laboratorio N°8

Objetivo del TP:

El objetivo de este trabajo práctico es que los alumnos realicen capturas con el analizador de red para estudiar diferentes protocolos. Las mismas se realizarán sobre una red interna del hogar.

Conocimientos Previos:

- Configuración de PC's
- Sistemas Operativos
- Aplicación Wireshark

Materiales necesarios:

- Computadoras personales con la aplicación Wireshark instalada
- Cliente FTP (Filezilla, winscp, o similar)

Síntesis de la tarea:

- Capturas con el analizador de protocolo.

Actividades a desarrollar:

- 1) Conexión y configuración de las PC's
- 2) Ejecución de comandos para verificar el funcionamiento de la red
 - Desde símbolo del sistema ejecutar los siguientes comandos
 - i. Ping a la dirección IP propia
 - ii. Ping a la dirección IP de una PC de otro grupo que está conectada o al default Gateway.
- 3) Capturar con el analizador de red denominado Wireshark los siguientes protocolos:
 - ICMP
 - i. PING, a una PC que se encuentre conectada. Realizar varias capturas modificando el tamaño del comando (por default, -l 1500 y -l 4000)
 - ii. TRACERT, a la misma PC del punto anterior.

➤ ARP

Sugerencia: Antes de comenzar a capturar con el analizador de protocolos, aplicar un filtro que solamente registre los paquetes cuya dirección IP coincida con la PC desde donde se está ejecutando los distintos comandos.

4) Pruebas hacia internet:

- Configurar en el Wireshark un filtro que solo capture el tráfico propio y capturar las siguientes conexiones para su análisis:
 - i. Sitio de la Universidad
 - ii. File Transfer Protocol (FTP) navegar y descargar un archivo pequeño de texto desde cualquier ftp de internet (p.ej. mirror.csclub.uwaterloo.ca)
- Quitar todos los filtros del Wireshark, comenzar a capturar por un período de tiempo (5 minutos), parar la captura, luego detallar y analizar todos los protocolos que aparecen en la misma.
- Sin filtro ejecutar el comando `ipconfig /release` para liberar la dirección IP de la placa de red y luego ejecutar el comando `ipconfig /renew` para renovar la misma.
 - i. Analizar los paquetes DHCP que se presentan.

5) Cuestionario

- a) Para que se utilizan y en qué nivel del modelo OSI se ubican los protocolos PING y TRACE
- b) Para que se utiliza el protocolo ARP, breve descripción del mismo.
- c) ¿Es posible leer desde el analizador de protocolo el texto descargado del servidor FTP? Mostrar evidencia de la respuesta.

6) Informe de los Alumnos por grupo

El informe debe describir con detalle y en el formato especificado las tareas realizadas en el laboratorio y responder el cuestionario.

INFORME

Nota: para la resolución de este trabajo práctico instalamos la herramienta Wireshark y el cliente FTP WinSCP.

Antes de arrancar con la actividad ejecutamos el comando **ipconfig** en el CMD para tener un poco más de contexto acerca de la máquina en donde se estaba por desarrollar todo el trabajo.

```
C:\Windows\System32>ipconfig

Windows IP Configuration

Ethernet adapter Ethernet:

    Connection-specific DNS Suffix  . : 
    IPv6 Address. . . . . : 2802:8010:a824:ba00:ff24:5e99:5026:86a
    Temporary IPv6 Address. . . . . : 2802:8010:a824:ba00:4e4:6728:54d5:e2a2
    Link-local IPv6 Address . . . . . : fe80::7f1f:6602:72bf:f088%13
    IPv4 Address. . . . . : 192.168.1.35
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : fe80::e6ab:89ff:feec:db2%13
                                192.168.1.1
```

PUNTO 2 – PING PONG

A continuación entramos a Wireshark, "Adapter for loopback traffic capture", y filtramos por ICMP. Luego ejecutamos los siguientes comandos:

ping 192.168.1.35

icmp						
No.	Time	Source	Destination	Protocol	Length	Info
6084	69.980602	192.168.1.35	192.168.1.35	ICMP	64	Echo (ping) request id=0x0001, seq=5/1280, ttl=128 (reply in 6085)
6085	69.980641	192.168.1.35	192.168.1.35	ICMP	64	Echo (ping) reply id=0x0001, seq=5/1280, ttl=128 (request in 6084)
6162	70.984379	192.168.1.35	192.168.1.35	ICMP	64	Echo (ping) request id=0x0001, seq=6/1536, ttl=128 (reply in 6166)
6166	70.984433	192.168.1.35	192.168.1.35	ICMP	64	Echo (ping) reply id=0x0001, seq=6/1536, ttl=128 (request in 6162)
6234	71.986860	192.168.1.35	192.168.1.35	ICMP	64	Echo (ping) request id=0x0001, seq=7/1792, ttl=128 (reply in 6235)
6235	71.986909	192.168.1.35	192.168.1.35	ICMP	64	Echo (ping) reply id=0x0001, seq=7/1792, ttl=128 (request in 6234)
6324	72.993340	192.168.1.35	192.168.1.35	ICMP	64	Echo (ping) request id=0x0001, seq=8/2048, ttl=128 (reply in 6325)
6325	72.993395	192.168.1.35	192.168.1.35	ICMP	64	Echo (ping) reply id=0x0001, seq=8/2048, ttl=128 (request in 6324)



```
> Frame 25106: 64 bytes on wire (512 bits), 64 bytes captured (512 bits) on interface \Device\NPF_{Loopback}, id 0
> Null/Loopback
> Internet Protocol Version 4, Src: 192.168.1.35, Dst: 192.168.1.35
✓ Internet Control Message Protocol
  Type: 0 (Echo (ping) reply)
  Code: 0
  Checksum: 0x554f [correct]
  [Checksum Status: Good]
  Identifier (BE): 1 (0x0001)
  Identifier (LE): 256 (0x0100)
  Sequence Number (BE): 12 (0x000c)
  Sequence Number (LE): 3072 (0x0c00)
  [Request frame: 25105]
  [Response time: 0.058 ms]
✓ Data (32 bytes)
  Data: 61626364656666768696a6b6c6d6e6f70717273747576776162636465666676869
  [Length: 32]
```

0000	02 00 00 00	45 00 00 3c	82 30 00 00	80 01 34 fa	...	E..< .0....4.
0010	c0 a8 01 23	c0 a8 01 23	00 00 55 4f	00 01 00 0c	...	#...# ..UO....
0020	61 62 63 64	65 66 67 68	69 6a 6b 6c	6d 6e 6f 70	abc	defgh ijklmnop
0030	71 72 73 74	75 76 77 61	62 63 64 65	66 67 68 69	qrstu	vwab cdefghi

Observación: en la sección de datos, al realizar el ping solo se ven las letras de la a-w, pero nunca x,y,z.

Esto ocurre porque es el patrón de texto por defecto que utiliza el comando `ping` en los sistemas operativos Windows (desde Windows XP) para rellenar el paquete. No es una falla de [Wireshark](#), sino el contenido real que viaja por la red. [Super User +3](#)

¿Por qué ese patrón exacto?

- 1. Relleno necesario:** El protocolo ICMP (usado por el *ping*) necesita un tamaño mínimo de datos para completar el tamaño estándar del paquete. [Super User +1](#)
- 2. El abecedario incompleto:** Windows utiliza un bloque de 32 bytes de datos repetitivos. Para completar exactamente esa longitud, el patrón `abcdefghijklmnopqrstuvwabcdefghijklmnop` se corta justo en la *w*, reiniciando el ciclo antes de llegar a la *x*, *y*, *z*. [Cisco Learning Network](#)
- 3. Propósito:** El texto en sí no importa. Son solo datos de relleno (*padding*) para verificar que el paquete completo viaja de un extremo a otro. [Super User +2](#)

En Wireshark, puedes comprobar esto seleccionando el paquete ICMP y mirando el panel *Packet Bytes* en la esquina inferior derecha, donde verás el abecedario cortado a la mitad en texto plano o hexadecimal. [Wireshark Q&A +2](#)

1. El búfer estático predefinido

En el código fuente de la herramienta `ping.exe` de Windows, el alfabeto no se genera dinámicamente letra por letra. Los desarrolladores escribieron en el código una cadena de texto estática con un tamaño fijo de **23 caracteres**, que es exactamente:

`"abcdefghijklmnopqrstuvw"` (de la 'a' a la 'w'). [Reddit: r/networking +2](#)

Decidieron detenerse en la *w* simplemente porque quisieron, o tal vez para que la cadena tuviera exactamente 23 bytes (un número primo) antes de empezar a

El "Secreto" del OS Fingerprinting

Esta peculiaridad técnica no es destructiva, pero sirve para hacer **OS Fingerprinting** (identificación de sistemas operativos). Analistas de seguridad y administradores de red saben que si un paquete ICMP viaja con el patrón truncado en la *w*, la máquina de origen es, sin lugar a dudas, un sistema Windows. Linux, por ejemplo, utiliza un tamaño por defecto diferente (56 o 64 bytes) y rellena sus paquetes de forma totalmente distinta (usando marcas de tiempo numéricas). [Super User +2](#)

Gemini nos indicó que si ejecutábamos el comando pero con un tamaño de 60 bytes, las letras aparecerían. Lo intentamos y no sucedió.

¿Qué encontró el profe sobre el asunto?

“Lo que si encontré fue una humorada que decía que a-w es como la abreviatura para after work (para después del trabajo), y como chiste pone que es una clara señal de que deberías optar por ir a tomar un whisky” (capo, marce, gracias por la data 😊)

En otros foros también se supuso que esto se podría deber a que se armó de esta manera para identificar de que sistema operativo de donde se realizó el ping. Sin embargo, esta teoría también se cae porque uno programando (en .Net, C# o en C) podría generar su propio comando ping y modificar el mensaje para que aparezca lo que uno quiere (pudiendo imitar, por ejemplo, esos 23 bytes de la a-w).

`ping 127.0.0.1`

34301	382.945342	127.0.0.1	127.0.0.1	ICMP	64 Echo (ping) request	id=0x0001, seq=13/3328, ttl=128 (reply in 34302)
34302	382.945378	127.0.0.1	127.0.0.1	ICMP	64 Echo (ping) reply	id=0x0001, seq=13/3328, ttl=128 (request in 34301)
34389	383.961326	127.0.0.1	127.0.0.1	ICMP	64 Echo (ping) request	id=0x0001, seq=14/3584, ttl=128 (reply in 34390)
34390	383.961382	127.0.0.1	127.0.0.1	ICMP	64 Echo (ping) reply	id=0x0001, seq=14/3584, ttl=128 (request in 34389)
34491	384.968063	127.0.0.1	127.0.0.1	ICMP	64 Echo (ping) request	id=0x0001, seq=15/3840, ttl=128 (reply in 34492)
34492	384.968107	127.0.0.1	127.0.0.1	ICMP	64 Echo (ping) reply	id=0x0001, seq=15/3840, ttl=128 (request in 34491)
34575	385.978062	127.0.0.1	127.0.0.1	ICMP	64 Echo (ping) request	id=0x0001, seq=16/4096, ttl=128 (reply in 34576)
34576	385.978106	127.0.0.1	127.0.0.1	ICMP	64 Echo (ping) reply	id=0x0001, seq=16/4096, ttl=128 (request in 34575)

¿Por qué realizamos ping con estas 2 direcciones IP?

La idea era analizar si surgía alguna diferencia entre ejecutar este comando hacia la dirección IP de clase C y el local host.

PUNTO 3 – PING CON TAMAÑO MODIFICADO

Volvimos a ejecutar el comando ping, pero ahora con el tamaño modificado en 1500 y 4000.

Capturas del ping de un paquete de 1500 bytes



```
C:\Windows\System32>ping 192.168.1.35 -l 1500

Pinging 192.168.1.35 with 1500 bytes of data:
Reply from 192.168.1.35: bytes=1500 time<1ms TTL=128
Reply from 192.168.1.35: bytes=1500 time<1ms TTL=128
Reply from 192.168.1.35: bytes=1500 time<1ms TTL=128
Reply from 192.168.1.35: bytes=1500 time<1ms TTL=128

Ping statistics for 192.168.1.35:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 0ms, Average = 0ms
```

```

> Frame 740: 1532 bytes on wire (12256 bits), 1532 bytes captured (12256 bits) on interface \Device\NPF_{Loopback}, id 0
> Null/Loopback
> Internet Protocol Version 4, Src: 192.168.1.35, Dst: 192.168.1.35
▼ Internet Control Message Protocol
    Type: 0 (Echo (ping) reply)
    Code: 0
    Checksum: 0x6c60 [correct]
    [Checksum Status: Good]
    Identifier (BE): 1 (0x0001)
    Identifier (LE): 256 (0x0100)
    Sequence Number (BE): 20 (0x0014)
    Sequence Number (LE): 5120 (0x1400)
    [Request frame: 739]
    [Response time: 0.066 ms]
▼ Data (1500 bytes)
    Data: 6162636465666768696a6b6c6d6e6f70717273747576776162636465666768696a6b6c6d...
    [Length: 1500]

```

0010	c0 a8 01 23 c0 a8 01 23 00 00 6c 60 00 01 00 14	...#...# ..l`... ..
0020	61 62 63 64 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70	abcdefghijklmnop
0030	71 72 73 74 75 76 77 61 62 63 64 65 66 67 68 69	qrstuvwxyzvw
0040	6a 6b 6c 6d 6e 6f 70 71 72 73 74 75 76 77 61 62	klmnopqrstu
0050	63 64 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70 71 72	cdefghijklmn
0060	73 74 75 76 77 61 62 63 64 65 66 67 68 69 6a 6b	stuvwxyzabcd
0070	6c 6d 6e 6f 70 71 72 73 74 75 76 77 61 62 63 64	lmnopqrstuvw
0080	65 66 67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74	efghijklmnop
0090	75 76 77 61 62 63 64 65 66 67 68 69 6a 6b 6c 6d	vwxyzabcde
00a0	6e 6f 70 71 72 73 74 75 76 77 61 62 63 64 65 66	nopqrstuvw
00b0	67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74 75 76	ghijklmnopqr
00c0	77 61 62 63 64 65 66 67 68 69 6a 6b 6c 6d 6e 6f	wxyzabcde
00d0	70 71 72 73 74 75 76 77 61 62 63 64 65 66 67 68	pqrstuvwxyz
00e0	69 6a 6b 6c 6d 6e 6f 70 71 72 73 74 75 76 77 61	ijklmnopqrst
00f0	62 63 64 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70 71	bcdefghijklm
0100	72 73 74 75 76 77 61 62 63 64 65 66 67 68 69 6a	rstuvwxyzabc
0110	6b 6c 6d 6e 6f 70 71 72 73 74 75 76 77 61 62 63	klmnopqrstuvw
0120	64 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73	defghijklmnop
0130	74 75 76 77 61 62 63 64 65 66 67 68 69 6a 6b 6c	tuvwxyzabcde
0140	6d 6e 6f 70 71 72 73 74 75 76 77 61 62 63 64 65	mnopqrstuvwxyz

Sequence Number (big endian representation) (icmp.seq), 2 bytes

6 | P á g i n a


```
C:\Windows\System32>ping 192.168.1.35 -l 4000

Pinging 192.168.1.35 with 4000 bytes of data:
Reply from 192.168.1.35: bytes=4000 time<1ms TTL=128
Reply from 192.168.1.35: bytes=4000 time<1ms TTL=128
Reply from 192.168.1.35: bytes=4000 time<1ms TTL=128
Reply from 192.168.1.35: bytes=4000 time<1ms TTL=128

Ping statistics for 192.168.1.35:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 0ms, Average = 0ms
```

No.	Time	Source	Destination	Protocol	Length	Info
73618	647.032385	192.168.1.35	192.168.1.35	ICMP	92	Echo (ping) request id=0x0001, seq=21/5376, ttl=128 (reply in 73619)
73619	647.032428	192.168.1.35	192.168.1.35	ICMP	92	Echo (ping) reply id=0x0001, seq=21/5376, ttl=128 (request in 73618)
73716	648.036318	192.168.1.35	192.168.1.35	ICMP	92	Echo (ping) request id=0x0001, seq=22/5632, ttl=128 (reply in 73717)
73717	648.036372	192.168.1.35	192.168.1.35	ICMP	92	Echo (ping) reply id=0x0001, seq=22/5632, ttl=128 (request in 73716)
73816	649.044353	192.168.1.35	192.168.1.35	ICMP	92	Echo (ping) request id=0x0001, seq=23/5888, ttl=128 (reply in 73817)
73817	649.044406	192.168.1.35	192.168.1.35	ICMP	92	Echo (ping) reply id=0x0001, seq=23/5888, ttl=128 (request in 73816)
73908	650.051070	192.168.1.35	192.168.1.35	ICMP	92	Echo (ping) request id=0x0001, seq=24/6144, ttl=128 (reply in 73909)
73909	650.051122	192.168.1.35	192.168.1.35	ICMP	92	Echo (ping) reply id=0x0001, seq=24/6144, ttl=128 (request in 73908)
77626	691.913777	192.168.1.35	192.168.1.35	ICMP	4032	Echo (ping) request id=0x0001, seq=25/6400, ttl=128 (reply in 77627)
77627	691.913829	192.168.1.35	192.168.1.35	ICMP	4032	Echo (ping) reply id=0x0001, seq=25/6400, ttl=128 (request in 77626)
77699	692.915576	192.168.1.35	192.168.1.35	ICMP	4032	Echo (ping) request id=0x0001, seq=26/6656, ttl=128 (reply in 77700)
77700	692.915652	192.168.1.35	192.168.1.35	ICMP	4032	Echo (ping) reply id=0x0001, seq=26/6656, ttl=128 (request in 77699)
77792	693.925524	192.168.1.35	192.168.1.35	ICMP	4032	Echo (ping) request id=0x0001, seq=27/6912, ttl=128 (reply in 77793)
77793	693.925598	192.168.1.35	192.168.1.35	ICMP	4032	Echo (ping) reply id=0x0001, seq=27/6912, ttl=128 (request in 77792)
77887	694.934934	192.168.1.35	192.168.1.35	ICMP	4032	Echo (ping) request id=0x0001, seq=28/7168, ttl=128 (reply in 77888)
77888	694.934962	192.168.1.35	192.168.1.35	ICMP	4032	Echo (ping) reply id=0x0001, seq=28/7168, ttl=128 (request in 77887)

>	Frame 77792: 4032 bytes on wire (32256 bits), 4032 bytes captured (32256 bits) on interface \Device\NPF_{...}, id 0
>	Null/Loopback
>	Internet Protocol Version 4, Src: 192.168.1.35, Dst: 192.168.1.35
>	Internet Control Message Protocol
>	Type: 8 (Echo (ping) request)
>	Code: 0
>	Checksum: 0xf2df [correct]
>	[Checksum Status: Good]
>	Identifier (BE): 1 (0x0001)
>	Identifier (LE): 256 (0x0100)
>	Sequence Number (BE): 27 (0x001b)
>	Sequence Number (LE): 6912 (0x1b00)
>	[Response frame: 77793]
>	Data (4000 bytes)
>	Data: 6162636465666768696a6b6c6d6e6f707172737475767778797a7b7c7d7e7f808182838485868788898a8b8c8d8e8f90919293949596979899a0a1a2a3a4a5a6a7a8a9aaabacadeaebecedeef
>	[Length: 4000]

0010	c0 a0 01 23 c0 a0 01 23 00 00 f2 df 00 01 00 11	...
0020	61 62 63 64 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70	abcdefghijklmnop
0030	71 72 73 74 75 76 77 78 79 7a 7b 7c 7d 7e 7f 80	qrstuvwxyz abcdefgh
0040	81 82 83 84 85 86 87 88 89 8a 8b 8c 8d 8e 8f 90	ijklmnopq rstuvwab
0050	91 92 93 94 95 96 97 98 99 a0 a1 a2 a3 a4 a5 a6	cdefghij klmnopqr
0060	a7 a8 a9 aa ab ac ad ae af b0 b1 b2 b3 b4 b5 b6	stuvwabc defghijk
0070	b7 b8 b9 ba bb bc bd be bf c0 c1 c2 c3 c4 c5 c6	lmnopqrs tuvabcd
0080	c7 c8 c9 ca cb cc cd ce cf d0 d1 d2 d3 d4 d5 d6	efghijkl mnopqrst
0090	d7 d8 d9 da db dc dd de df e0 e1 e2 e3 e4 e5 e6	vwxyz abcdefghijklm
00a0	e7 e8 e9 ea eb ec ed ee ef f0 f1 f2 f3 f4 f5 f6	nopqrstu vwabcde
00b0	f7 f8 f9 fa fb fc fd fe ff 00 01 02 03 04 05 06	ghijklmn opqrstuv
00c0	07 08 09 0a 0b 0c 0d 0e 0f 10 11 12 13 14 15 16	wxyz abcdefghijklmno
00d0	17 18 19 1a 1b 1c 1d 1e 1f 20 21 22 23 24 25 26	pqrstuvw abcdefgh
00e0	27 28 29 2a 2b 2c 2d 2e 2f 30 31 32 33 34 35 36	ijklmnop qrstuvw
00f0	37 38 39 3a 3b 3c 3d 3e 3f 40 41 42 43 44 45 46	bcdefghijklmnopq
0100	47 48 49 4a 4b 4c 4d 4e 4f 50 51 52 53 54 55 56	rstuvwab cdefghij
0110	57 58 59 5a 5b 5c 5d 5e 5f 60 61 62 63 64 65 66	klmnopqr stuvwabc
0120	67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74 75 76	defghijkl mnopqrs
0130	77 78 79 7a 7b 7c 7d 7e 7f 80 81 82 83 84 85 86	tuvwabcd efghijkl
0140	87 88 89 8a 8b 8c 8d 8e 8f 90 91 92 93 94 95 96	mnopqrst uvwabcde

(En este punto nos colgamos en sacar captura de pantalla y terminamos sacando captura del fragmento del video que grabamos resolviendo el TP, por eso la mala calidad)

Observación: En paquetes de 1500 bytes, el tiempo de respuesta es de aproximadamente 0,065 milisegundos. En cambio, al realizar el ping en un paquete de 4000 bytes, este tiempo aumenta a 0,074 milisegundos.

PUNTO 3 – TRACERT Y ARP

El paso siguiente fue ejecutar el comando **tracert 192.168.1.35**

```
C:\Windows\System32>tracert 192.168.1.35

Tracing route to DESKTOP-987BVBN [192.168.1.35]
over a maximum of 30 hops:

  1    <1 ms    <1 ms    <1 ms    DESKTOP-987BVBN [192.168.1.35]

Trace complete.
```

Observación: Que el máximo sea “30 hops” significa que el comando **tracert** rastreará la ruta de los paquetes de red hacia su destino pasando como máximo por 30 routers (saltos) antes de dar por terminada la búsqueda.

88157	810.532614	192.168.1.35	192.168.1.35	ICMP	96 Echo (ping) request	id=0x0001, seq=29/7424, ttl=1 (reply in 88158)
88158	810.532648	192.168.1.35	192.168.1.35	ICMP	96 Echo (ping) reply	id=0x0001, seq=29/7424, ttl=128 (request in 88157)
88159	810.532913	192.168.1.35	192.168.1.35	ICMP	96 Echo (ping) request	id=0x0001, seq=30/7680, ttl=1 (reply in 88160)
88160	810.532938	192.168.1.35	192.168.1.35	ICMP	96 Echo (ping) reply	id=0x0001, seq=30/7680, ttl=128 (request in 88159)
88161	810.533179	192.168.1.35	192.168.1.35	ICMP	96 Echo (ping) request	id=0x0001, seq=31/7936, ttl=1 (reply in 88162)
88162	810.533203	192.168.1.35	192.168.1.35	ICMP	96 Echo (ping) reply	id=0x0001, seq=31/7936, ttl=128 (request in 88161)

> Frame 77888: 4032 bytes on wire (32256 bits), 4032 bytes captured (32256 bits) on interface \Device\NPF_{...}, id 0 > Null/Loopback > Internet Protocol Version 4, Src: 192.168.1.35, Dst: 192.168.1.35 0100 = Version: 4 0101 = Header Length: 20 bytes (5) > Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT) Total Length: 4028 Identification: 0x8248 (33352) > Flags: 0x00 Fragment Offset: 0 Time to Live: 128 Protocol: ICMP (1) Header Checksum: 0x2562 [validation disabled] [Header checksum status: Unverified] Source Address: 192.168.1.35 Destination Address: 192.168.1.35 > Internet Control Message Protocol Type: 0 (Echo (ping) reply) Code: 0 Checksum: 0xfade [correct] [Checksum Status: Good] Identifier (BE): 1 (0x0001) Identifier (LE): 256 (0x0100) Sequence Number (BE): 28 (0x001c) Sequence Number (LE): 7168 (0x1c00) [Request frame: 77887] [Response time: 0.048 ms] > Data (4000 bytes) Data: 6162636465666768696a6b6c6d6e6f707172737475767768696a6b6c6d... [Length: 4000]	
--	--

TTL = 128; LENGHT = 4000; DATA = VARIADA

88157	810.532614	192.168.1.35	192.168.1.35	ICMP	96 Echo (ping) request	id=0x0001, seq=29/7424, ttl=1 (reply in 88158)
88158	810.532648	192.168.1.35	192.168.1.35	ICMP	96 Echo (ping) reply	id=0x0001, seq=29/7424, ttl=128 (request in 88157)
88159	810.532913	192.168.1.35	192.168.1.35	ICMP	96 Echo (ping) request	id=0x0001, seq=30/7680, ttl=1 (reply in 88160)
88160	810.532938	192.168.1.35	192.168.1.35	ICMP	96 Echo (ping) reply	id=0x0001, seq=30/7680, ttl=128 (request in 88159)
88161	810.533179	192.168.1.35	192.168.1.35	ICMP	96 Echo (ping) request	id=0x0001, seq=31/7936, ttl=1 (reply in 88162)
88162	810.533203	192.168.1.35	192.168.1.35	ICMP	96 Echo (ping) reply	id=0x0001, seq=31/7936, ttl=128 (request in 88161)

> Frame 88157: 96 bytes on wire (768 bits), 96 bytes captured (768 bits) on interface \Device\NPF_{...}, id 0 > Null/Loopback > Internet Protocol Version 4, Src: 192.168.1.35, Dst: 192.168.1.35 0100 = Version: 4 0101 = Header Length: 20 bytes (5) > Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT) Total Length: 92 Identification: 0x8249 (33353) > Flags: 0x00 Fragment Offset: 0 Time to Live: 1 [Expert Info (Note/Sequence): "Time To Live" only 1] ["Time To Live" only 1] [Severity level: Note] [Group: Sequence] Protocol: ICMP (1) Header Checksum: 0xb3c1 [validation disabled] [Header checksum status: Unverified] Source Address: 192.168.1.35 Destination Address: 192.168.1.35 > Internet Control Message Protocol Type: 8 (Echo (ping) request) Code: 0 Checksum: 0xf7e1 [correct] [Checksum Status: Good] Identifier (BE): 1 (0x0001) Identifier (LE): 256 (0x0100) Sequence Number (BE): 29 (0x001d) Sequence Number (LE): 7424 (0x1d00) [Response frame: 88158] > Data (64 bytes) Data: 00... [Length: 64]	
--	--

TTL = 1; LENGHT = 64; DATA = 0

Notamos el color celeste que resaltaba la sección de IPv4. La desplegamos y descubrimos que el TTL también estaba resaltado con el mismo color. Sin embargo existía una diferencia esencial:

Al lanzar un **request**, el TTL es 1. → Es el paquete de diagnóstico real que viaja hacia la red. Wireshark puede desplegar información adicional o advertencias porque ese paquete está a punto de expirar y es relevante para el funcionamiento de tracer.

Al recibir un **reply**, el TTL es 128. → Pertenece a la propia computadora, enviando una respuesta ICMP falsa de Wireshark para armar la gráfica. Normalmente solo muestra el valor y no hay nada más que explicar.

Además, mientras los **request** tenían un tamaño de 64 bytes y estaba todo lleno de ceros, los **reply** poseían un tamaño de 4000 bytes y sus valores de relleno eran variados.

Observación: luego notamos que este patrón de “tamaños y rellenos” no siempre se cumplía, por que aparecían **reply** vacíos (todo cero) y **request** llenos (datos variados).

León Della Paolera 7:43 PM

por que en wireshark, al hacer tracer en reply el ttl marca 128 sin desplegar, en cambio en recibido marca ttl = 1. Esto ocurre debido a una confusión muy común al interpretar las dos columnas de dirección en Wireshark (origen/destino) y el comportamiento nativo del comando tracer en Windows. El resumen rápido es: El TTL de 128 pertenece a tu propia computadora enviando la respuesta ICMP falsa de Wireshark para armar la gráfica, mientras que el TTL = 1 es el paquete de diagnóstico real que viaja hacia la red. El desglose exacto de por qué ves esto en Wireshark ocurre por las siguientes razones:

1. El paquete "Recibido" (TTL = 1) es la consulta saliente. Aunque Wireshark lo categorice de forma confusa en su flujo, el paquete que marca TTL = 1 es el paquete de diagnóstico original (Echo Request o UDP) que genera tu comando tracer.

2. El paquete "Reply" (TTL = 128) es el comportamiento por defecto de Windows

Cuando ves un paquete en Wireshark con TTL = 128, estás viendo el valor inicial estándar que el sistema operativo Windows le asigna a cualquier paquete IP nuevo que genera.

- En este escenario, la fila de "Reply" con TTL 128 no significa que el enrutador te respondió con ese número.
- Significa que el sistema operativo emisor (tu PC o el destino final si es un servidor Windows) armó un paquete desde cero. Como el valor por defecto en Windows es 128, el encabezado IP se rellena con ese número antes de ser enviado a la red.
- Al no estar "desplegado" o decrementado, te confirma que estás viendo el paquete justo en su punto de origen (origen puro sin haber cruzado ningún enrutador).

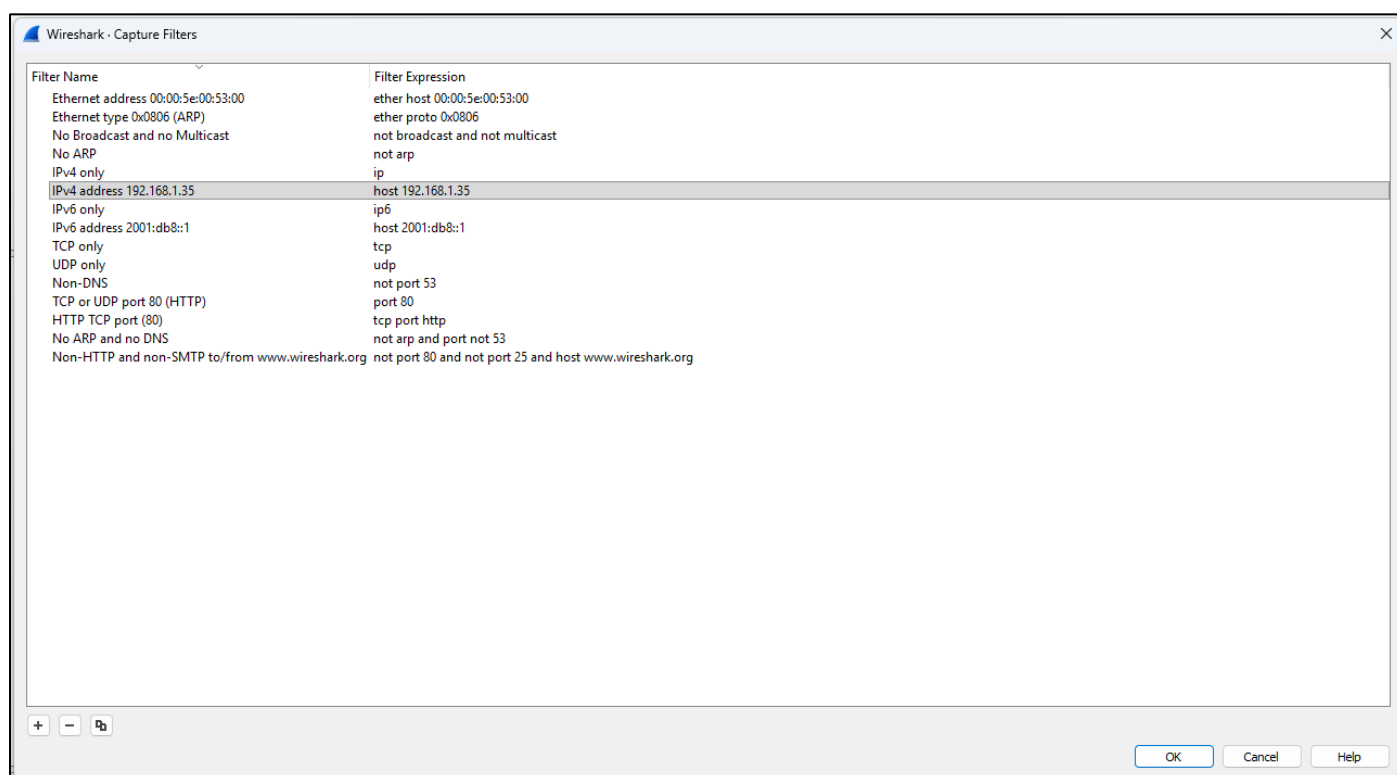
ARP

Continuamos ejecutando el comando ARP. Primero limpiamos la tabla con un **arp -d*** y luego **arp -a**.



```
C:\Windows\System32>arp -a

Interface: 192.168.1.35 --- 0xd
Internet Address      Physical Address      Type
192.168.1.1           e4-ab-89-ec-0d-b2    dynamic
192.168.1.49          78-66-9d-4c-e7-4c    dynamic
192.168.1.255         ff-ff-ff-ff-ff-ff    static
224.0.0.2             01-00-5e-00-00-02    static
224.0.0.22            01-00-5e-00-00-16    static
224.0.0.251           01-00-5e-00-00-fb    static
224.0.0.252           01-00-5e-00-00-fc    static
239.255.255.250       01-00-5e-7f-ff-fa    static
255.255.255.255       ff-ff-ff-ff-ff-ff    static
```



Tuvimos complicaciones al inicio porque en el Wireshark estábamos filtrando por una dirección IP de la máquina cuando deberíamos haber filtrado únicamente por protocolo ARP (y/o por MAC Address), porque es un protocolo de capa 2 y las direcciones origen y destino están en hexadecimal (como las direcciones MAC).

Esta es la conversación que tuvieron:

(Paquetes 1245 - 1246)

Router (al equipo 1.35): "¿Quién de ustedes tiene la IP 192.168.1.35? Pásenle el dato a la IP 192.168.1.1 (que soy yo)."

La PC: "¡Soy yo! Mi dirección física (MAC) es 60:cf:84:ca:1e:b0."

(Paquetes 9111 - 9113)

La PC (al Router): "¿Quién tiene la IP 192.168.1.1? Pásenle el dato a la IP 192.168.1.35."



Router: "¡Acá estoy! Mi dirección física (MAC) es e4:ab:89:ec:0d:b2."

(Paquetes 15907 - 15908)

La PC (Gritando a todos - Broadcast): "¡HOLA A TODOS! ¿Alguien me dice quién tiene la IP del router (192.168.1.1)? Se me olvidó o necesito confirmar."

Router (Respondiéndote de forma directa): "Sigo siendo yo. Te repito, mi MAC es e4:ab:89:ec:0d:b2."

(Paquetes 16727 - 16728)

La PC (Gritando a todos - Broadcast): "¡HOLA! Necesito hablar con otra persona. ¿Quién de ustedes tiene la IP 192.168.1.49? Avisen a la IP 1.35."

Otro equipo (Respondiéndote directo): "¡Ah, ese soy yo! Mi MAC es 78:66:9d:4c:e7:4c."

(Paquetes 19114 en adelante...)

Router y La PC: (Se la pasan preguntándose periódicamente "¿Che, 1.35, seguís teniendo esa MAC?" y "¿Che, 1.1, seguís ahí?").

Básicamente, a partir de acá, ambos equipos están refrescando sus memorias (la tabla caché ARP). Como los registros ARP tienen un tiempo de vida corto (para evitar enviar datos a un equipo que ya se desconectó), se siguen "haciendo ping" con ARP cada ciertos segundos para confirmar que el otro sigue vivo y conectado en la misma dirección.

No.	Time	Source	Destination	Protocol	Length	Info
1245	7.296797	MitraSta_ec:0d:b2	60:cf:84:ca:1e:b0	ARP	60	Who has 192.168.1.35? Tell 192.168.1.1
1246	7.296805	60:cf:84:ca:1e:b0	MitraSta_ec:0d:b2	ARP	42	192.168.1.35 is at 60:cf:84:ca:1e:b0
9111	41.191560	60:cf:84:ca:1e:b0	MitraSta_ec:0d:b2	ARP	42	Who has 192.168.1.1? Tell 192.168.1.35
9113	41.191768	MitraSta_ec:0d:b2	60:cf:84:ca:1e:b0	ARP	60	192.168.1.1 is at e4:ab:89:ec:0d:b2
15907	67.717935	60:cf:84:ca:1e:b0	Broadcast	ARP	42	Who has 192.168.1.1? Tell 192.168.1.35
15908	67.718466	MitraSta_ec:0d:b2	60:cf:84:ca:1e:b0	ARP	60	192.168.1.1 is at e4:ab:89:ec:0d:b2
16727	71.202902	60:cf:84:ca:1e:b0	Broadcast	ARP	42	Who has 192.168.1.49? Tell 192.168.1.35
16728	71.206187	78:66:9d:4c:e7:4c	60:cf:84:ca:1e:b0	ARP	60	192.168.1.49 is at 78:66:9d:4c:e7:4c
19114	80.042020	MitraSta_ec:0d:b2	60:cf:84:ca:1e:b0	ARP	60	Who has 192.168.1.35? Tell 192.168.1.1
19115	80.042024	60:cf:84:ca:1e:b0	MitraSta_ec:0d:b2	ARP	42	192.168.1.35 is at 60:cf:84:ca:1e:b0
31088	123.690895	60:cf:84:ca:1e:b0	MitraSta_ec:0d:b2	ARP	42	Who has 192.168.1.1? Tell 192.168.1.35
31089	123.691076	MitraSta_ec:0d:b2	60:cf:84:ca:1e:b0	ARP	60	192.168.1.1 is at e4:ab:89:ec:0d:b2
33772	135.716198	MitraSta_ec:0d:b2	60:cf:84:ca:1e:b0	ARP	60	Who has 192.168.1.35? Tell 192.168.1.1
33773	135.716206	60:cf:84:ca:1e:b0	MitraSta_ec:0d:b2	ARP	42	192.168.1.35 is at 60:cf:84:ca:1e:b0
40817	165.196428	60:cf:84:ca:1e:b0	MitraSta_ec:0d:b2	ARP	42	Who has 192.168.1.1? Tell 192.168.1.35
40818	165.196602	MitraSta_ec:0d:b2	60:cf:84:ca:1e:b0	ARP	60	192.168.1.1 is at e4:ab:89:ec:0d:b2
45337	185.119335	MitraSta_ec:0d:b2	60:cf:84:ca:1e:b0	ARP	60	Who has 192.168.1.35? Tell 192.168.1.1
45338	185.119345	60:cf:84:ca:1e:b0	MitraSta_ec:0d:b2	ARP	42	192.168.1.35 is at 60:cf:84:ca:1e:b0
63735	255.719940	MitraSta_ec:0d:b2	60:cf:84:ca:1e:b0	ARP	60	Who has 192.168.1.35? Tell 192.168.1.1
63736	255.719945	60:cf:84:ca:1e:b0	MitraSta_ec:0d:b2	ARP	42	192.168.1.35 is at 60:cf:84:ca:1e:b0
77332	315.713970	MitraSta_ec:0d:b2	60:cf:84:ca:1e:b0	ARP	60	Who has 192.168.1.35? Tell 192.168.1.1
77333	315.713978	60:cf:84:ca:1e:b0	MitraSta_ec:0d:b2	ARP	42	192.168.1.35 is at 60:cf:84:ca:1e:b0
80658	330.192199	60:cf:84:ca:1e:b0	MitraSta_ec:0d:b2	ARP	42	Who has 192.168.1.1? Tell 192.168.1.35

¿Por qué aparece es MitraSta_ec:0d:b2?

León Della Paolera 8:19 PM

Que una dirección MAC empiece por **mitraSta_ec** (o similar) se debe a que **Wireshark** traduce automáticamente los primeros 3 bytes de la dirección MAC por el nombre del fabricante del hardware. [DNS Checker](#)

Ese texto es la abreviatura de **MitraStar Technology Corporation**, un gigante asiático que fabrica routers, módems y puntos de acceso para grandes operadoras. [MAC Address Lookup](#)

Modem router de movistar

Capturas ARP desplegado

1245 7.296797	MitraSta_ec:0d:b2	60:cf:84:ca:1e:b0	ARP	60 Who has 192.168.1.35? Tell 192.168.1.1
1246 7.296805	60:cf:84:ca:1e:b0	MitraSta_ec:0d:b2	ARP	42 192.168.1.35 is at 60:cf:84:ca:1e:b0
9111 41.191560	60:cf:84:ca:1e:b0	MitraSta_ec:0d:b2	ARP	42 Who has 192.168.1.1? Tell 192.168.1.35
9113 41.191768	MitraSta_ec:0d:b2	60:cf:84:ca:1e:b0	ARP	60 192.168.1.1 is at e4:ab:89:ec:0d:b2
15907 67.717935	60:cf:84:ca:1e:b0	Broadcast	ARP	42 Who has 192.168.1.1? Tell 192.168.1.35
15908 67.718466	MitraSta_ec:0d:b2	60:cf:84:ca:1e:b0	ARP	60 192.168.1.1 is at e4:ab:89:ec:0d:b2
16727 71.202902	60:cf:84:ca:1e:b0	Broadcast	ARP	42 Who has 192.168.1.49? Tell 192.168.1.35
16728 71.206187	78:66:9d:4c:e7:4c	60:cf:84:ca:1e:b0	ARP	60 192.168.1.49 is at 78:66:9d:4c:e7:4c
19114 80.042020	MitraSta_ec:0d:b2	60:cf:84:ca:1e:b0	ARP	60 Who has 192.168.1.35? Tell 192.168.1.1
19115 80.042024	60:cf:84:ca:1e:b0	MitraSta_ec:0d:b2	ARP	42 192.168.1.35 is at 60:cf:84:ca:1e:b0
31088 123.690895	60:cf:84:ca:1e:b0	MitraSta_ec:0d:b2	ARP	42 Who has 192.168.1.1? Tell 192.168.1.35
31089 123.691076	MitraSta_ec:0d:b2	60:cf:84:ca:1e:b0	ARP	60 192.168.1.1 is at e4:ab:89:ec:0d:b2

> Frame 1245: 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface \Device\NPF_{9F68F6EF-CF68-4DDB-A7A6-F8C24DE4B225}, id 0

▼ Ethernet II, Src: MitraSta_ec:0d:b2 (e4:ab:89:ec:0d:b2), Dst: 60:cf:84:ca:1e:b0 (60:cf:84:ca:1e:b0)

> Destination: 60:cf:84:ca:1e:b0 (60:cf:84:ca:1e:b0)

> Source: MitraSta_ec:0d:b2 (e4:ab:89:ec:0d:b2)

Type: ARP (0x0806)

Padding: 00000000000000000000000000000000

▼ Address Resolution Protocol (request)

Hardware type: Ethernet (1)

Protocol type: IPv4 (0x0800)

Hardware size: 6

Protocol size: 4

Opcode: request (1)

Sender MAC address: MitraSta_ec:0d:b2 (e4:ab:89:ec:0d:b2)

Sender IP address: 192.168.1.1

Target MAC address: 00:00:00:00:00:00 (00:00:00:00:00:00)

Target IP address: 192.168.1.35

1246 7.296805	60:cf:84:ca:1e:b0	MitraSta_ec:0d:b2	ARP	42 192.168.1.35 is at 60:cf:84:ca:1e:b0
9111 41.191560	60:cf:84:ca:1e:b0	MitraSta_ec:0d:b2	ARP	42 Who has 192.168.1.1? Tell 192.168.1.35
9113 41.191768	MitraSta_ec:0d:b2	60:cf:84:ca:1e:b0	ARP	60 192.168.1.1 is at e4:ab:89:ec:0d:b2
15907 67.717935	60:cf:84:ca:1e:b0	Broadcast	ARP	42 Who has 192.168.1.1? Tell 192.168.1.35
15908 67.718466	MitraSta_ec:0d:b2	60:cf:84:ca:1e:b0	ARP	60 192.168.1.1 is at e4:ab:89:ec:0d:b2
16727 71.202902	60:cf:84:ca:1e:b0	Broadcast	ARP	42 Who has 192.168.1.49? Tell 192.168.1.35
16728 71.206187	78:66:9d:4c:e7:4c	60:cf:84:ca:1e:b0	ARP	60 192.168.1.49 is at 78:66:9d:4c:e7:4c
19114 80.042020	MitraSta_ec:0d:b2	60:cf:84:ca:1e:b0	ARP	60 Who has 192.168.1.35? Tell 192.168.1.1
19115 80.042024	60:cf:84:ca:1e:b0	MitraSta_ec:0d:b2	ARP	42 192.168.1.35 is at 60:cf:84:ca:1e:b0
31088 123.690895	60:cf:84:ca:1e:b0	MitraSta_ec:0d:b2	ARP	42 Who has 192.168.1.1? Tell 192.168.1.35
31089 123.691076	MitraSta_ec:0d:b2	60:cf:84:ca:1e:b0	ARP	60 192.168.1.1 is at e4:ab:89:ec:0d:b2

> Frame 1246: 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface \Device\NPF_{9F68F6EF-CF68-4DDB-A7A6-F8C24DE4B225}, id 0

▼ Ethernet II, Src: 60:cf:84:ca:1e:b0 (60:cf:84:ca:1e:b0), Dst: MitraSta_ec:0d:b2 (e4:ab:89:ec:0d:b2)

> Destination: MitraSta_ec:0d:b2 (e4:ab:89:ec:0d:b2)

> Source: 60:cf:84:ca:1e:b0 (60:cf:84:ca:1e:b0)

Type: ARP (0x0806)

▼ Address Resolution Protocol (reply)

Hardware type: Ethernet (1)

Protocol type: IPv4 (0x0800)

Hardware size: 6

Protocol size: 4

Opcode: reply (2)

Sender MAC address: 60:cf:84:ca:1e:b0 (60:cf:84:ca:1e:b0)

Sender IP address: 192.168.1.35

Target MAC address: MitraSta_ec:0d:b2 (e4:ab:89:ec:0d:b2)

Target IP address: 192.168.1.1

9111 41.191560	60:cf:84:ca:1e:b0	MitraSta_ec:0d:b2	ARP	42 Who has 192.168.1.1? Tell 192.168.1.35
9113 41.191768	MitraSta_ec:0d:b2	60:cf:84:ca:1e:b0	ARP	60 192.168.1.1 is at e4:ab:89:ec:0d:b2
15907 67.717935	60:cf:84:ca:1e:b0	Broadcast	ARP	42 Who has 192.168.1.1? Tell 192.168.1.35
15908 67.718466	MitraSta_ec:0d:b2	60:cf:84:ca:1e:b0	ARP	60 192.168.1.1 is at e4:ab:89:ec:0d:b2
16727 71.202902	60:cf:84:ca:1e:b0	Broadcast	ARP	42 Who has 192.168.1.49? Tell 192.168.1.35
16728 71.206187	78:66:9d:4c:e7:4c	60:cf:84:ca:1e:b0	ARP	60 192.168.1.49 is at 78:66:9d:4c:e7:4c
19114 80.042020	MitraSta_ec:0d:b2	60:cf:84:ca:1e:b0	ARP	60 Who has 192.168.1.35? Tell 192.168.1.1
19115 80.042024	60:cf:84:ca:1e:b0	MitraSta_ec:0d:b2	ARP	42 192.168.1.35 is at 60:cf:84:ca:1e:b0
31088 123.690895	60:cf:84:ca:1e:b0	MitraSta_ec:0d:b2	ARP	42 Who has 192.168.1.1? Tell 192.168.1.35
31089 123.691076	MitraSta_ec:0d:b2	60:cf:84:ca:1e:b0	ARP	60 192.168.1.1 is at e4:ab:89:ec:0d:b2

> Frame 9111: 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface \Device\NPF_{9F68F6EF-CF68-4DDB-A7A6-F8C24DE4B225}, id 0

▼ Ethernet II, Src: 60:cf:84:ca:1e:b0 (60:cf:84:ca:1e:b0), Dst: MitraSta_ec:0d:b2 (e4:ab:89:ec:0d:b2)

> Destination: MitraSta_ec:0d:b2 (e4:ab:89:ec:0d:b2)

> Source: 60:cf:84:ca:1e:b0 (60:cf:84:ca:1e:b0)

Type: ARP (0x0806)

▼ Address Resolution Protocol (request)

Hardware type: Ethernet (1)

Protocol type: IPv4 (0x0800)

Hardware size: 6

Protocol size: 4

Opcode: request (1)

Sender MAC address: 60:cf:84:ca:1e:b0 (60:cf:84:ca:1e:b0)

Sender IP address: 192.168.1.35

Target MAC address: MitraSta_ec:0d:b2 (e4:ab:89:ec:0d:b2)

Target IP address: 192.168.1.1

9113	41.191768	MitraSta_ec:0d:b2	60:cf:84:ca:1e:b0	ARP	60	192.168.1.1 is at e4:ab:89:ec:0d:b2
15907	67.717935	60:cf:84:ca:1e:b0	Broadcast	ARP	42	Who has 192.168.1.1? Tell 192.168.1.35
15908	67.718466	MitraSta_ec:0d:b2	60:cf:84:ca:1e:b0	ARP	60	192.168.1.1 is at e4:ab:89:ec:0d:b2
16727	71.202902	60:cf:84:ca:1e:b0	Broadcast	ARP	42	Who has 192.168.1.49? Tell 192.168.1.35
16728	71.206187	78:66:9d:4c:e7:4c	60:cf:84:ca:1e:b0	ARP	60	192.168.1.49 is at 78:66:9d:4c:e7:4c
19114	80.042020	MitraSta_ec:0d:b2	60:cf:84:ca:1e:b0	ARP	60	Who has 192.168.1.35? Tell 192.168.1.1
19115	80.042024	60:cf:84:ca:1e:b0	MitraSta_ec:0d:b2	ARP	42	192.168.1.35 is at 60:cf:84:ca:1e:b0
31088	123.690895	60:cf:84:ca:1e:b0	MitraSta_ec:0d:b2	ARP	42	Who has 192.168.1.1? Tell 192.168.1.35
31089	123.691076	MitraSta_ec:0d:b2	60:cf:84:ca:1e:b0	ARP	60	192.168.1.1 is at e4:ab:89:ec:0d:b2

> Frame 9113: 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface \Device\NPF_{9F68F6EF-CF6B-4DDB-A7A6-F8C24DE4B225}, id 0

▼ Ethernet II, Src: MitraSta_ec:0d:b2 (e4:ab:89:ec:0d:b2), Dst: 60:cf:84:ca:1e:b0 (60:cf:84:ca:1e:b0)

> Destination: 60:cf:84:ca:1e:b0 (60:cf:84:ca:1e:b0)

> Source: MitraSta_ec:0d:b2 (e4:ab:89:ec:0d:b2)

Type: ARP (0x0806)

Padding: 00000000000000000000000000000000

▼ Address Resolution Protocol (reply)

Hardware type: Ethernet (1)

Protocol type: IPv4 (0x0800)

Hardware size: 6

Protocol size: 4

Opcode: reply (2)

Sender MAC address: MitraSta_ec:0d:b2 (e4:ab:89:ec:0d:b2)

Sender IP address: 192.168.1.1

Target MAC address: 60:cf:84:ca:1e:b0 (60:cf:84:ca:1e:b0)

Target IP address: 192.168.1.35

16727	71.202902	60:cf:84:ca:1e:b0	Broadcast	ARP	42 Who has 192.168.1.49? Tell 192.168.1.35
16728	71.206187	78:66:9d:4c:e7:4c	60:cf:84:ca:1e:b0	ARP	60 192.168.1.49 is at 78:66:9d:4c:e7:4c
19114	80.042020	MitraSta_ec:0d:b2	60:cf:84:ca:1e:b0	ARP	60 Who has 192.168.1.35? Tell 192.168.1.1
19115	80.042024	60:cf:84:ca:1e:b0	MitraSta_ec:0d:b2	ARP	42 192.168.1.35 is at 60:cf:84:ca:1e:b0
31088	123.690895	60:cf:84:ca:1e:b0	MitraSta_ec:0d:b2	ARP	42 Who has 192.168.1.1? Tell 192.168.1.35
31089	123.691076	MitraSta_ec:0d:b2	60:cf:84:ca:1e:b0	ARP	60 192.168.1.1 is at e4:ab:89:ec:0d:b2

> Frame 16727: 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface \Device\NPF_{9F68F6EF-CF6B-4DDB-A7A6-F8C24DE4B225}, id 0

▼ Ethernet II, Src: 60:cf:84:ca:1e:b0 (60:cf:84:ca:1e:b0), Dst: Broadcast (ff:ff:ff:ff:ff:ff)

 > Destination: Broadcast (ff:ff:ff:ff:ff:ff)

 > Source: 60:cf:84:ca:1e:b0 (60:cf:84:ca:1e:b0)

 Type: ARP (0x0806)

▼ Address Resolution Protocol (request)

 Hardware type: Ethernet (1)

 Protocol type: IPv4 (0x0800)

 Hardware size: 6

 Protocol size: 4

 Opcode: request (1)

 Sender MAC address: 60:cf:84:ca:1e:b0 (60:cf:84:ca:1e:b0)

 Sender IP address: 192.168.1.35

 Target MAC address: 00:00:00_00:00:00 (00:00:00:00:00:00)

 Target IP address: 192.168.1.49

16728	71.206187	78:66:9d:4c:e7:4c	60:cf:84:ca:1e:b0	ARP	60	192.168.1.49 is at 78:66:9d:4c:e7:4c
19114	80.042020	MitraSta_ec:0d:b2	60:cf:84:ca:1e:b0	ARP	60	Who has 192.168.1.35? Tell 192.168.1.1
19115	80.042024	60:cf:84:ca:1e:b0	MitraSta_ec:0d:b2	ARP	42	192.168.1.35 is at 60:cf:84:ca:1e:b0
31088	123.690895	60:cf:84:ca:1e:b0	MitraSta_ec:0d:b2	ARP	42	Who has 192.168.1.1? Tell 192.168.1.35
31089	123.691076	MitraSta_ec:0d:b2	60:cf:84:ca:1e:b0	ARP	60	192.168.1.1 is at e4:ab:89:ec:0d:b2

> Frame 16728: 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface \Device\NPF_{9F68F6EF-CF6B-4DD8-A7A6-F8C24DE4B225}, id 0

▼ Ethernet II, Src: 78:66:9d:4c:e7:4c (78:66:9d:4c:e7:4c), Dst: 60:cf:84:ca:1e:b0 (60:cf:84:ca:1e:b0)

> Destination: 60:cf:84:ca:1e:b0 (60:cf:84:ca:1e:b0)

> Source: 78:66:9d:4c:e7:4c (78:66:9d:4c:e7:4c)

Type: ARP (0x0806)

Padding: 00000000000000000000000000000000

▼ Address Resolution Protocol (reply)

Hardware type: Ethernet (1)

Protocol type: IPv4 (0x0800)

Hardware size: 6

Protocol size: 4

Opcode: reply (2)

Sender MAC address: 78:66:9d:4c:e7:4c (78:66:9d:4c:e7:4c)

Sender IP address: 192.168.1.49

Target MAC address: 60:cf:84:ca:1e:b0 (60:cf:84:ca:1e:b0)

Target IP address: 192.168.1.35

Observación: Cuando el Router o la máquina con MAC Address “78:66:9d:4c:e7:4c” realiza una consulta, aparece un padding lleno de ceros. En contraposición, cuando analizamos las consultas del propio equipo, este padding no aparece.

Resulta que el campo padding en ARP es un relleno de bytes (generalmente ceros) que se añade para cumplir con el tamaño mínimo de una trama Ethernet (64 bytes), ya que los paquetes ARP por sí solos no alcanzan dicho tamaño.



León Della Paolera 8:24 PM

En el contexto de redes (como en los paquetes ARP que estás analizando), el padding se utiliza para **completar el tamaño mínimo obligatorio** que exige un protocolo para poder transmitir los datos.

¿Por qué se necesita relleno en los paquetes ARP?

1. **La regla de Ethernet:** El protocolo Ethernet (Capa 2) exige que cualquier trama que viaje por el cable tenga un tamaño mínimo de **64 bytes** (incluyendo la cabecera y el chequeo de errores). Si restamos la cabecera, los datos puros deben medir al menos **46 bytes**.
2. **El problema de ARP:** Un paquete ARP (Capa 3) es muy pequeño. Solo contiene direcciones IP y MAC, por lo que su tamaño real es de apenas **28 bytes**.

Si algunas respuestas poseen padding y otras no **¿Por qué el tamaño de la pregunta no es el mismo que en la respuesta?**

Según Gemini: En realidad, la diferencia de tamaño no ocurre en el cable de red, sino que es una especie de "ilusión óptica" generada por el punto exacto en el que Wireshark captura el tráfico. En el medio físico, tanto la pregunta como la respuesta miden exactamente lo mismo.

La discrepancia entre los 42 bytes y los 60 bytes se debe a si el paquete es de salida (creado por tu PC) o de entrada (creado por otro equipo):

Tus propios paquetes ARP (42 bytes): Esto es tráfico saliente. Wireshark intercepta estos paquetes desde la pila de red de tu sistema operativo **antes** de que lleguen al hardware de tu tarjeta de red (NIC). En este punto, el paquete está "crudo": tiene los 14 bytes de la cabecera Ethernet más los 28 bytes del mensaje ARP, sumando 42 bytes. Es tu propia tarjeta de red física la que, una fracción de segundo después, le inyecta el padding de ceros para cumplir el estándar de Ethernet antes de soltarlo al cable. Como Wireshark captura el dato antes de ese paso por el hardware, no ve el relleno.

Paquetes del Router o del otro equipo (60 bytes): Esto es tráfico entrante. Estos paquetes ya viajaron por el cable físico. La tarjeta de red del equipo emisor ya hizo su trabajo y les agregó el padding correspondiente. Cuando tu tarjeta de red los recibe, los procesa y se los pasa a Wireshark con el relleno incluido, por lo que logras ver esos ceros adicionales hasta llegar a los 60 bytes.

En resumen: si pudieras analizar el cable a nivel físico, verías que todos los paquetes ARP alcanzan el tamaño mínimo de 64 bytes (los 60 bytes que ves en el tráfico entrante + 4 bytes de verificación de errores o FCS, que las tarjetas de red suelen descartar antes de pasarle la captura a Wireshark).



PUNTO 4 – PRUEBAS HACIA INTERNET

Entramos al portal de la U.N.O. y en el CMD realizamos un ping al sitio.

León Della Paolera 8:40 PM

```

C:\Users\lldel>curl campusvirtual.uno.edu.ar
<html><body><h1>It works!</h1>
<p>This is the default web page for this server.</p>
<p>The web server software is running but no content has been added, yet.</p>
</body></html>

C:\Users\lldel>
        
```

Gonza 8:40 PM

```

C:\Users\Gonzalo>ping uno.edu.ar

Haciendo ping a uno.edu.ar [144.217.88.40] con 32 bytes de datos:
Respuesta desde 144.217.88.40: bytes=32 tiempo=168ms TTL=46
Respuesta desde 144.217.88.40: bytes=32 tiempo=173ms TTL=46
Respuesta desde 144.217.88.40: bytes=32 tiempo=174ms TTL=46
Respuesta desde 144.217.88.40: bytes=32 tiempo=172ms TTL=46

Estadísticas de ping para 144.217.88.40:
    Paquetes: enviados = 4, recibidos = 4, perdidos = 0
    (0% perdidos),
    Tiempos aproximados de ida y vuelta en milisegundos:
        Mínimo = 168ms, Máximo = 174ms, Media = 171ms
        
```

En Wireshark filtramos por la dirección IP de la máquina y por `tcp.port==443`.

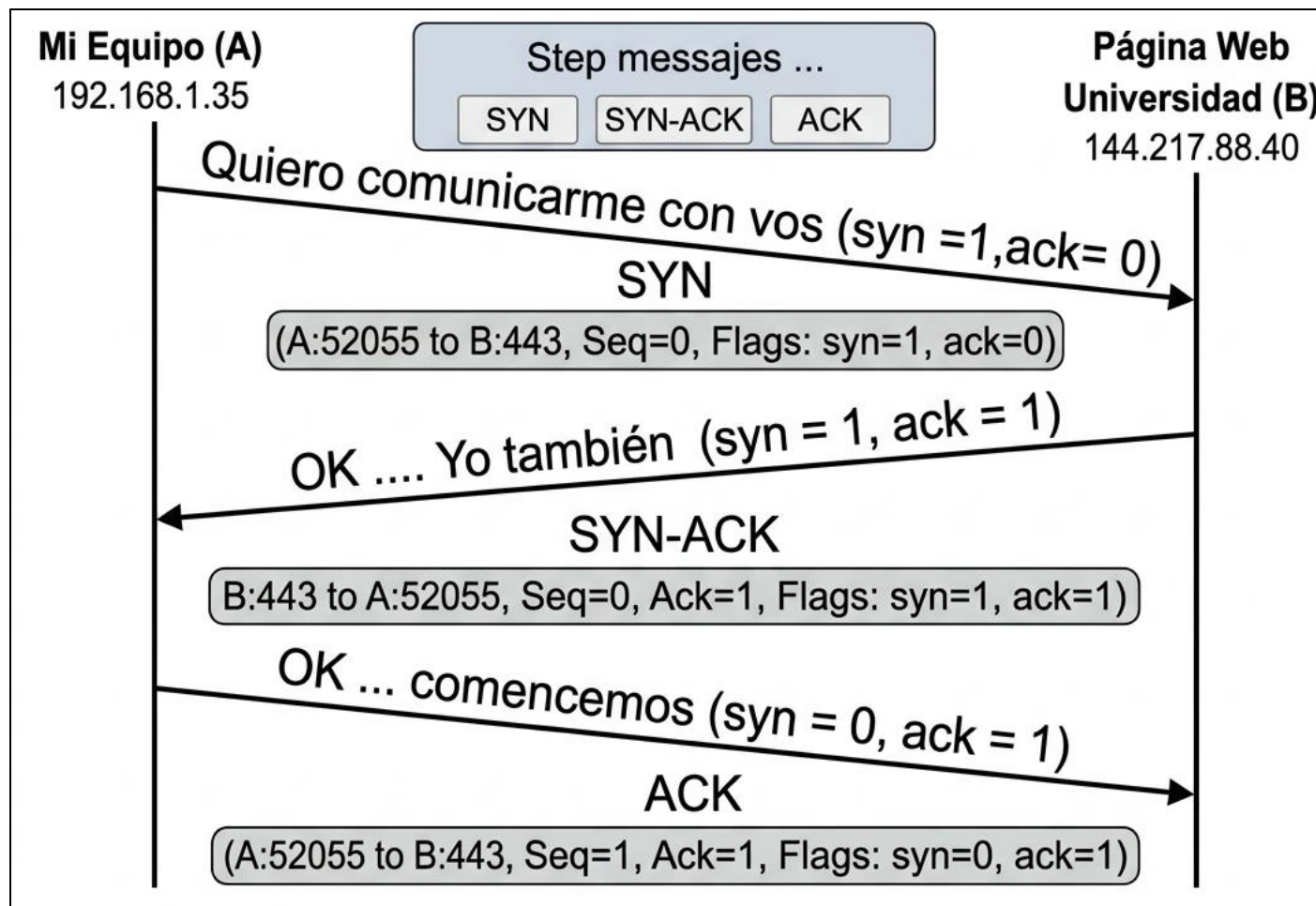
Automáticamente nos lanzó estos resultados:

ip.addr==192.168.1.35 and tcp.port==443						
No.	Time	Source	Destination	Protocol	Length	Info
348	1.723848	192.168.1.35	144.217.88.40	TLSv1.2	595	Application Data
368	1.890540	144.217.88.40	192.168.1.35	TCP	60	443 → 52046 [ACK] Seq=1 Ack=542 Win=313 Len=0
465	2.719285	144.217.88.40	192.168.1.35	TCP	1494	443 → 52046 [ACK] Seq=1 Ack=542 Win=313 Len=1440 [TCP segment of a reassembled PDU]
466	2.719285	144.217.88.40	192.168.1.35	TCP	1494	443 → 52046 [ACK] Seq=1441 Ack=542 Win=313 Len=1440 [TCP segment of a reassembled PDU]
467	2.719319	192.168.1.35	144.217.88.40	TCP	54	52046 → 443 [ACK] Seq=542 Ack=2881 Win=511 Len=0
468	2.719330	144.217.88.40	192.168.1.35	TCP	1494	443 → 52046 [ACK] Seq=2881 Ack=542 Win=313 Len=1440 [TCP segment of a reassembled PDU]
469	2.719330	144.217.88.40	192.168.1.35	TCP	1494	443 → 52046 [ACK] Seq=4321 Ack=542 Win=313 Len=1440 [TCP segment of a reassembled PDU]
470	2.719330	144.217.88.40	192.168.1.35	TCP	1494	443 → 52046 [ACK] Seq=5761 Ack=542 Win=313 Len=1440 [TCP segment of a reassembled PDU]
471	2.719340	192.168.1.35	144.217.88.40	TCP	54	52046 → 443 [ACK] Seq=542 Ack=7201 Win=511 Len=0
472	2.719345	144.217.88.40	192.168.1.35	TCP	1494	443 → 52046 [ACK] Seq=7201 Ack=542 Win=313 Len=1440 [TCP segment of a reassembled PDU]
473	2.719345	144.217.88.40	192.168.1.35	TCP	1494	443 → 52046 [ACK] Seq=8641 Ack=542 Win=313 Len=1440 [TCP segment of a reassembled PDU]
474	2.719354	192.168.1.35	144.217.88.40	TCP	54	52046 → 443 [ACK] Seq=542 Ack=10081 Win=511 Len=0
475	2.719359	144.217.88.40	192.168.1.35	TCP	1494	443 → 52046 [ACK] Seq=10081 Ack=542 Win=313 Len=1440 [TCP segment of a reassembled PDU]
476	2.719366	192.168.1.35	144.217.88.40	TCP	54	52046 → 443 [ACK] Seq=542 Ack=11521 Win=511 Len=0
477	2.719370	144.217.88.40	192.168.1.35	TCP	1494	443 → 52046 [ACK] Seq=11521 Ack=542 Win=313 Len=1440 [TCP segment of a reassembled PDU]
478	2.719378	144.217.88.40	192.168.1.35	TCP	1494	443 → 52046 [ACK] Seq=12961 Ack=542 Win=313 Len=1440 [TCP segment of a reassembled PDU]
479	2.719383	192.168.1.35	144.217.88.40	TCP	54	52046 → 443 [ACK] Seq=542 Ack=14401 Win=511 Len=0
499	2.885060	144.217.88.40	192.168.1.35	TCP	1494	443 → 52046 [ACK] Seq=14401 Ack=542 Win=313 Len=1440 [TCP segment of a reassembled PDU]
500	2.885060	144.217.88.40	192.168.1.35	TLSv1.2	1494	Application Data [TCP segment of a reassembled PDU]
501	2.885088	192.168.1.35	144.217.88.40	TCP	54	52046 → 443 [ACK] Seq=542 Ack=17281 Win=511 Len=0
502	2.885098	144.217.88.40	192.168.1.35	TCP	1494	443 → 52046 [ACK] Seq=17281 Ack=542 Win=313 Len=1440 [TCP segment of a reassembled PDU]
503	2.885098	144.217.88.40	192.168.1.35	TCP	1494	443 → 52046 [ACK] Seq=18721 Ack=542 Win=313 Len=1440 [TCP segment of a reassembled PDU]
504	2.885108	192.168.1.35	144.217.88.40	TCP	54	52046 → 443 [ACK] Seq=542 Ack=20161 Win=511 Len=0



Diagrama del camino de 3 vías

Realizamos un boceto de los datos que nos tiró el analizador de protocolo.



Capturas de analizador de protocolo

574	4.185656	192.168.1.35	144.217.88.40	TCP	66	52055 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM=1
613	4.342685	144.217.88.40	192.168.1.35	TCP	66	443 → 52055 [SYN, ACK] Seq=0 Ack=1 Win=29200 Len=0 MSS=1440 SACK_PERM=1 WS=128
614	4.342722	192.168.1.35	144.217.88.40	TCP	54	52055 → 443 [ACK] Seq=1 Ack=1 Win=65280 Len=0
615	4.343054	192.168.1.35	144.217.88.40	TLS	19	Client Hello
649	4.500956	144.217.88.40	192.168.1.35	TCP	60	443 → 52055 [ACK] Seq=1 Ack=1441 Win=32128 Len=0
650	4.501010	144.217.88.40	192.168.1.35	TCP	60	443 → 52055 [ACK] Seq=1 Ack=1896 Win=35072 Len=0
652	4.502339	144.217.88.40	192.168.1.35	TLS	14	Server Hello, Change Cipher Spec, Application Data
653	4.502339	144.217.88.40	192.168.1.35	TLS	14	Application Data [TCP segment of a reassembled PDU]
654	4.502339	144.217.88.40	192.168.1.35	TLS	324	Application Data, Application Data
655	4.502372	192.168.1.35	144.217.88.40	TCP	54	52055 → 443 [ACK] Seq=1896 Ack=3151 Win=65280 Len=0
656	4.503392	192.168.1.35	144.217.88.40	TLS	134	Change Cipher Spec, Application Data
657	4.503525	192.168.1.35	144.217.88.40	TLS	638	Application Data

Frame 574: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface \Device\NPF_{9F68F6EF-CF6B-40DB-A7A6-F8C24DE4B225}, id 0
 Ethernet II, Src: 60:cf:84:ca:1e:b0 (60:cf:84:ca:1e:b0), Dst: MitraSta_ec:0d:b2 (e4:ab:89:ec:0d:b2)
 Internet Protocol Version 4, Src: 192.168.1.35, Dst: 144.217.88.40
 Transmission Control Protocol, Src Port: 52055, Dst Port: 443, Seq: 0, Len: 0
 Source Port: 52055
 Destination Port: 443
 [Stream index: 2]
 [TCP Segment Len: 0]
 Sequence Number: 0 (relative sequence number)
 Sequence Number (raw): 50050425
 [Next Sequence Number: 1 (relative sequence number)]
 Acknowledgment Number: 0
 Acknowledgment number (raw): 0
 1000 = Header Length: 32 bytes (8)
 Flags: 0x002 (SYN)
 000. = Reserved: Not set
 ...0 = Nonce: Not set
0... = Congestion Window Reduced (CWR): Not set
0... = ECN-Echo: Not set
0... = Urgent: Not set
0... = Acknowledgment: Not set
0... = Push: Not set

574	4.185656	192.168.1.35	144.217.88.40	TCP	66 52055 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM=1
613	4.342685	144.217.88.40	192.168.1.35	TCP	66 443 → 52055 [SYN, ACK] Seq=0 Ack=1 Win=29200 Len=0 MSS=1440 SACK_PERM=1 WS=128
614	4.342722	192.168.1.35	144.217.88.40	TCP	54 52055 → 443 [ACK] Seq=1 Ack=1 Win=65280 Len=0
615	4.343054	192.168.1.35	144.217.88.40	TLS...	19... Client Hello
649	4.500956	144.217.88.40	192.168.1.35	TCP	60 443 → 52055 [ACK] Seq=1 Ack=1441 Win=32128 Len=0
650	4.501010	144.217.88.40	192.168.1.35	TCP	60 443 → 52055 [ACK] Seq=1 Ack=1896 Win=35072 Len=0
652	4.502339	144.217.88.40	192.168.1.35	TLS...	14... Server Hello, Change Cipher Spec, Application Data
653	4.502339	144.217.88.40	192.168.1.35	TLS...	14... Application Data [TCP segment of a reassembled PDU]
654	4.502339	144.217.88.40	192.168.1.35	TLS...	324 Application Data, Application Data
655	4.502372	192.168.1.35	144.217.88.40	TCP	54 52055 → 443 [ACK] Seq=1896 Ack=3151 Win=65280 Len=0
656	4.503392	192.168.1.35	144.217.88.40	TLS...	134 Change Cipher Spec, Application Data
657	4.503525	192.168.1.35	144.217.88.40	TLS...	638 Application Data

> Frame 613: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface \Device\NPF_{9F68F6EF-CF6B-4DDB-A7A6-F8C24DE4B225}

> Ethernet II, Src: MitraSta_ec:0d:b2 (e4:ab:89:ec:0d:b2), Dst: 60:cf:84:ca:1e:b0 (60:cf:84:ca:1e:b0)

> Internet Protocol Version 4, Src: 144.217.88.40, Dst: 192.168.1.35

✓ Transmission Control Protocol, Src Port: 443, Dst Port: 52055, Seq: 0, Ack: 1, Len: 0

Source Port: 443

Destination Port: 52055

[Stream index: 2]

[TCP Segment Len: 0]

Sequence Number: 0 (relative sequence number)

Sequence Number (raw): 1623607470

[Next Sequence Number: 1 (relative sequence number)]

Acknowledgment Number: 1 (relative ack number)

Acknowledgment number (raw): 50050426

1000 = Header Length: 32 bytes (8)

✓ Flags: 0x012 (SYN, ACK)

000. = Reserved: Not set

...0 = Nonce: Not set

.... 0... = Congestion Window Reduced (CWR): Not set

.... .0.. = ECN-Echo: Not set

.... ..0. = Urgent: Not set

.... ...1 = Acknowledgment: Set

.... 0... = Push: Not set

614	4.342722	192.168.1.35	144.217.88.40	TCP	54 52055 → 443 [ACK] Seq=1 Ack=1 Win=65280 Len=0
615	4.343054	192.168.1.35	144.217.88.40	TLS...	19... Client Hello
649	4.500956	144.217.88.40	192.168.1.35	TCP	60 443 → 52055 [ACK] Seq=1 Ack=1441 Win=32128 Len=0
650	4.501010	144.217.88.40	192.168.1.35	TCP	60 443 → 52055 [ACK] Seq=1 Ack=1896 Win=35072 Len=0
652	4.502339	144.217.88.40	192.168.1.35	TLS...	14... Server Hello, Change Cipher Spec, Application Data
653	4.502339	144.217.88.40	192.168.1.35	TLS...	14... Application Data [TCP segment of a reassembled PDU]
654	4.502339	144.217.88.40	192.168.1.35	TLS...	324 Application Data, Application Data
655	4.502372	192.168.1.35	144.217.88.40	TCP	54 52055 → 443 [ACK] Seq=1896 Ack=3151 Win=65280 Len=0
656	4.503392	192.168.1.35	144.217.88.40	TLS...	134 Change Cipher Spec, Application Data
657	4.503525	192.168.1.35	144.217.88.40	TLS...	638 Application Data

> Frame 614: 54 bytes on wire (432 bits), 54 bytes captured (432 bits) on interface \Device\NPF_{9F68F6EF-CF6B-4DDB-A7A6-F8C24DE4B225}, id 0

> Ethernet II, Src: 60:cf:84:ca:1e:b0 (60:cf:84:ca:1e:b0), Dst: MitraSta_ec:0d:b2 (e4:ab:89:ec:0d:b2)

> Internet Protocol Version 4, Src: 192.168.1.35, Dst: 144.217.88.40

✓ Transmission Control Protocol, Src Port: 52055, Dst Port: 443, Seq: 1, Ack: 1, Len: 0

Source Port: 52055

Destination Port: 443

[Stream index: 2]

[TCP Segment Len: 0]

Sequence Number: 1 (relative sequence number)

Sequence Number (raw): 50050426

[Next Sequence Number: 1 (relative sequence number)]

Acknowledgment Number: 1 (relative ack number)

Acknowledgment number (raw): 1623607471

0101 = Header Length: 20 bytes (5)

✓ Flags: 0x010 (ACK)

000. = Reserved: Not set

...0 = Nonce: Not set

.... 0... = Congestion Window Reduced (CWR): Not set

.... .0.. = ECN-Echo: Not set

.... ..0. = Urgent: Not set

.... ...1 = Acknowledgment: Set

.... 0... = Push: Not set



615	4.343054	192.168.1.35	144.217.88.40	TLS...	19...	Client Hello
649	4.500956	144.217.88.40	192.168.1.35	TCP	60 443 → 52055	[ACK] Seq=1 Ack=1441 Win=32128 Len=0
650	4.501010	144.217.88.40	192.168.1.35	TCP	60 443 → 52055	[ACK] Seq=1 Ack=1896 Win=35072 Len=0
652	4.502339	144.217.88.40	192.168.1.35	TLS...	14...	Server Hello, Change Cipher Spec, Application Data
653	4.502339	144.217.88.40	192.168.1.35	TLS...	14...	Application Data [TCP segment of a reassembled PDU]
654	4.502339	144.217.88.40	192.168.1.35	TLS...	324	Application Data, Application Data
655	4.502372	192.168.1.35	144.217.88.40	TCP	54 52055 → 443	[ACK] Seq=1896 Ack=3151 Win=65280 Len=0
656	4.503392	192.168.1.35	144.217.88.40	TLS...	134	Change Cipher Spec, Application Data
657	4.503525	192.168.1.35	144.217.88.40	TLS...	638	Application Data
> Frame 615: 1949 bytes on wire (15592 bits), 1949 bytes captured (15592 bits) on interface \Device\NPF_{9F68F6EF-CF6B-4DDB-A7A6-F8C24DE4B225}, id 0 > Ethernet II, Src: 60:cf:84:ca:1e:b0 (60:cf:84:ca:1e:b0), Dst: MitraSta_ec:0d:b2 (e4:ab:89:ec:0d:b2) > Internet Protocol Version 4, Src: 192.168.1.35, Dst: 144.217.88.40 > Transmission Control Protocol, Src Port: 52055, Dst Port: 443, Seq: 1, Ack: 1, Len: 1895 Source Port: 52055 Destination Port: 443 [Stream index: 2] [TCP Segment Len: 1895] Sequence Number: 1 (relative sequence number) Sequence Number (raw): 50050426 [Next Sequence Number: 1896 (relative sequence number)] Acknowledgment Number: 1 (relative ack number) Acknowledgment number (raw): 1623607471 0101 = Header Length: 20 bytes (5) Flags: 0x018 (PSH, ACK) 000. = Reserved: Not set ...0 = Nonce: Not set ... 0... = Congestion Window Reduced (CWR): Not set0.. = ECN-Echo: Not set0. = Urgent: Not set1 = Acknowledgment: Set 1... = Push: Set						
649	4.500956	144.217.88.40	192.168.1.35	TCP	60 443 → 52055	[ACK] Seq=1 Ack=1441 Win=32128 Len=0
650	4.501010	144.217.88.40	192.168.1.35	TCP	60 443 → 52055	[ACK] Seq=1 Ack=1896 Win=35072 Len=0
652	4.502339	144.217.88.40	192.168.1.35	TLS...	14...	Server Hello, Change Cipher Spec, Application Data
653	4.502339	144.217.88.40	192.168.1.35	TLS...	14...	Application Data [TCP segment of a reassembled PDU]
654	4.502339	144.217.88.40	192.168.1.35	TLS...	324	Application Data, Application Data
655	4.502372	192.168.1.35	144.217.88.40	TCP	54 52055 → 443	[ACK] Seq=1896 Ack=3151 Win=65280 Len=0
656	4.503392	192.168.1.35	144.217.88.40	TLS...	134	Change Cipher Spec, Application Data
657	4.503525	192.168.1.35	144.217.88.40	TLS...	638	Application Data
> Frame 649: 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface \Device\NPF_{9F68F6EF-CF6B-4DDB-A7A6-F8C24DE4B225}, id 0 > Ethernet II, Src: MitraSta_ec:0d:b2 (e4:ab:89:ec:0d:b2), Dst: 60:cf:84:ca:1e:b0 (60:cf:84:ca:1e:b0) > Internet Protocol Version 4, Src: 144.217.88.40, Dst: 192.168.1.35 > Transmission Control Protocol, Src Port: 443, Dst Port: 52055, Seq: 1, Ack: 1441, Len: 0 Source Port: 443 Destination Port: 52055 [Stream index: 2] [TCP Segment Len: 0] Sequence Number: 1 (relative sequence number) Sequence Number (raw): 1623607471 [Next Sequence Number: 1 (relative sequence number)] Acknowledgment Number: 1441 (relative ack number) Acknowledgment number (raw): 50051866 0101 = Header Length: 20 bytes (5) Flags: 0x010 (ACK) 000. = Reserved: Not set ...0 = Nonce: Not set ... 0... = Congestion Window Reduced (CWR): Not set0.. = ECN-Echo: Not set0. = Urgent: Not set1 = Acknowledgment: Set 0... = Push: Not set						

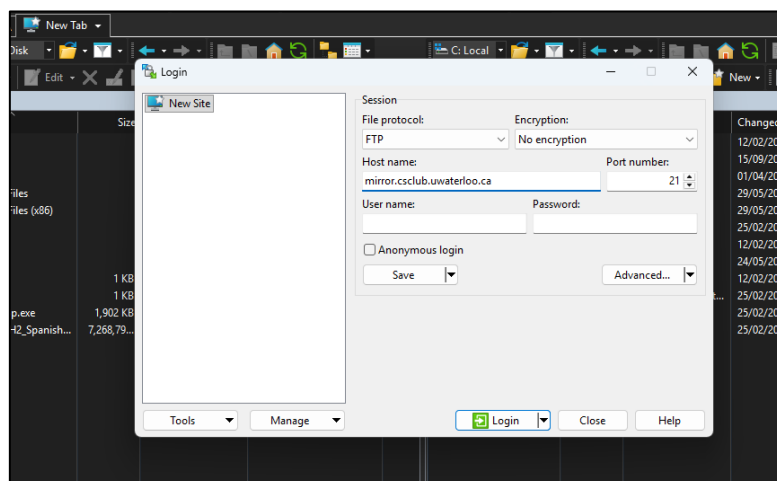


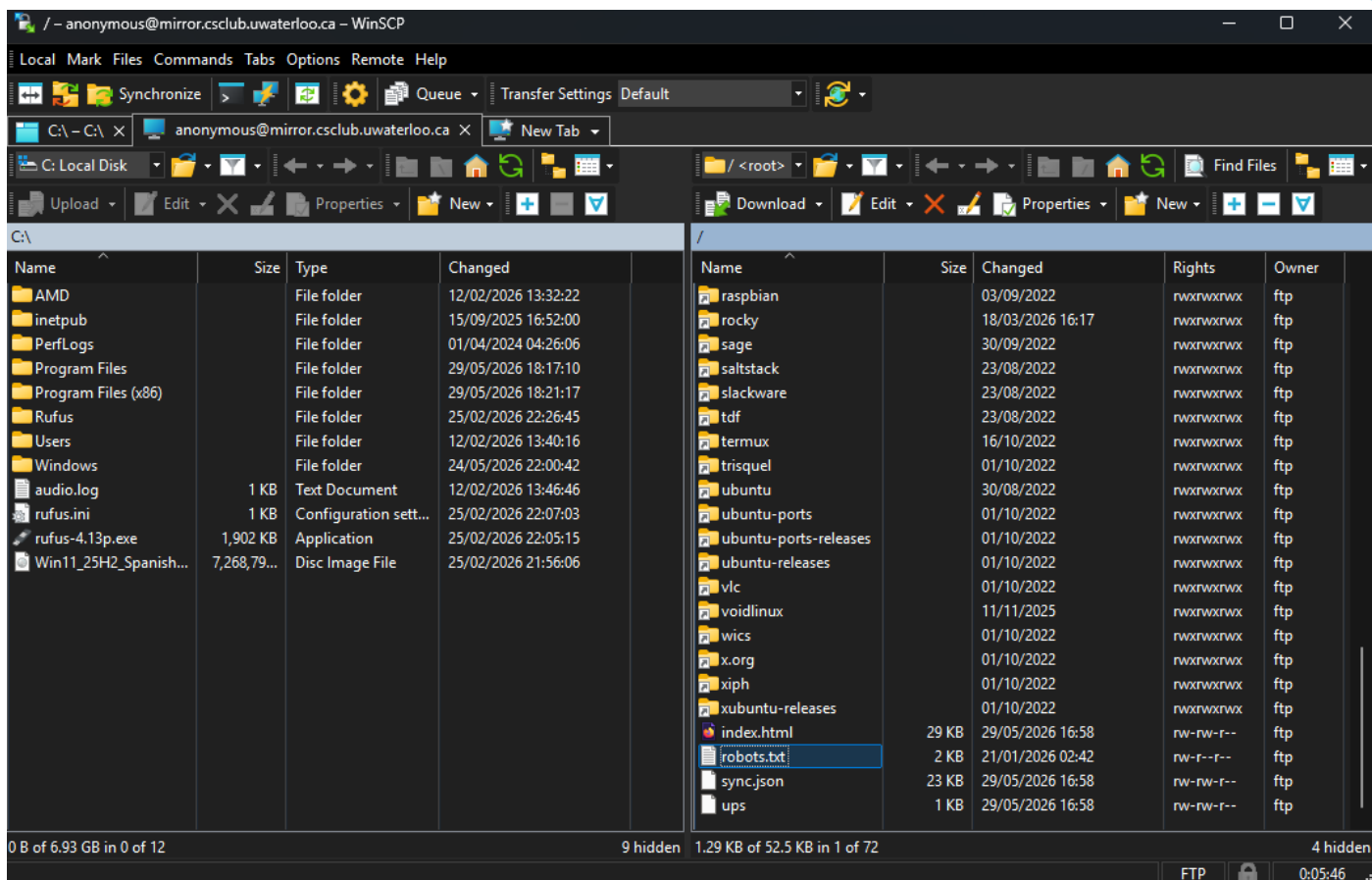
```
Acknowledgment Number: 0
Acknowledgment number (raw): 0
1000 .... = Header Length: 32 bytes (8)
v Flags: 0x002 (SYN)
  000. .... = Reserved: Not set
  ...0 .... = Nonce: Not set
  .... 0... = Congestion Window Reduced (CWR): Not set
  .... .0.. = ECN-Echo: Not set
  .... ..0. = Urgent: Not set
  .... ...0 = Acknowledgment: Not set
  .... .... 0... = Push: Not set
  .... .... .0.. = Reset: Not set
  > .... .... ..1. = Syn: Set
  .... .... ...0 = Fin: Not set
  [TCP Flags: .....S.]
Window: 65535
[Calculated window size: 65535]
Checksum: 0xaaf3 [unverified]
[Checksum Status: Unverified]
Urgent Pointer: 0
> Options: (12 bytes), Maximum segment size, No-Operation (NOP), Window scale, No-Operation (NOP), No-Operation (NOP), SACK permitted
> [Timestamps]
```

```
Acknowledgment Number: 1 (relative ack number)
Acknowledgment number (raw): 50050426
1000 .... = Header Length: 32 bytes (8)
v Flags: 0x012 (SYN, ACK)
  000. .... = Reserved: Not set
  ...0 .... = Nonce: Not set
  .... 0... = Congestion Window Reduced (CWR): Not set
  .... .0.. = ECN-Echo: Not set
  .... ..0. = Urgent: Not set
  .... ...1 = Acknowledgment: Set
  .... .... 0... = Push: Not set
  .... .... .0.. = Reset: Not set
  > .... .... ..1. = Syn: Set
  .... .... ...0 = Fin: Not set
  [TCP Flags: .....A..S.]
Window: 29200
[Calculated window size: 29200]
Checksum: 0x233b [unverified]
[Checksum Status: Unverified]
Urgent Pointer: 0
> Options: (12 bytes), Maximum segment size, No-Operation (NOP), No-Operation (NOP), SACK permitted, No-Operation (NOP), Window scale
> [SEQ/ACK analysis]
```

PUNTO 4 – FTP

A través de WINSXP entramos como usuario anónimo a la página indicada y descargamos el archivo “robots”.





566 4.629441	2802:8010:a824:ba00:4e4:6728:54d5:e2a2	2620:101:f000:4901:c5c:0:f:1055	FTP	79 Request: PMD
590 4.784435	2620:101:f000:4901:c5c:0:f:1055	2802:8010:a824:ba00:4e4:6728:54d5:e2a2	FTP	108 Response: 257 "/" is the current directory
591 4.784589	2802:8010:a824:ba00:4e4:6728:54d5:e2a2	2620:101:f000:4901:c5c:0:f:1055	FTP	91 Request: CMD /robots.txt
605 4.939387	2620:101:f000:4901:c5c:0:f:1055	2802:8010:a824:ba00:4e4:6728:54d5:e2a2	FTP	107 Response: 550 Failed to change directory.
606 4.939564	2802:8010:a824:ba00:4e4:6728:54d5:e2a2	2620:101:f000:4901:c5c:0:f:1055	FTP	82 Request: TYPE I
617 5.094412	2620:101:f000:4901:c5c:0:f:1055	2802:8010:a824:ba00:4e4:6728:54d5:e2a2	FTP	105 Response: 200 Switching to Binary mode.
618 5.094557	2802:8010:a824:ba00:4e4:6728:54d5:e2a2	2620:101:f000:4901:c5c:0:f:1055	FTP	92 Request: SIZE /robots.txt
632 5.249563	2620:101:f000:4901:c5c:0:f:1055	2802:8010:a824:ba00:4e4:6728:54d5:e2a2	FTP	84 Response: 213 1324
633 5.249693	2802:8010:a824:ba00:4e4:6728:54d5:e2a2	2620:101:f000:4901:c5c:0:f:1055	FTP	92 Request: MDTM /robots.txt
647 5.404678	2620:101:f000:4901:c5c:0:f:1055	2802:8010:a824:ba00:4e4:6728:54d5:e2a2	FTP	94 Response: 213 20260121054249
654 5.498536	2802:8010:a824:ba00:4e4:6728:54d5:e2a2	2620:101:f000:4901:c5c:0:f:1055	FTP	82 Request: TYPE I
667 5.653446	2620:101:f000:4901:c5c:0:f:1055	2802:8010:a824:ba00:4e4:6728:54d5:e2a2	FTP	105 Response: 200 Switching to Binary mode.
668 5.653556	2802:8010:a824:ba00:4e4:6728:54d5:e2a2	2620:101:f000:4901:c5c:0:f:1055	FTP	80 Request: EPSV
713 5.809439	2620:101:f000:4901:c5c:0:f:1055	2802:8010:a824:ba00:4e4:6728:54d5:e2a2	FTP	122 Response: 229 Entering Extended Passive Mode (6292)
714 5.809686	2802:8010:a824:ba00:4e4:6728:54d5:e2a2	2620:101:f000:4901:c5c:0:f:1055	FTP	91 Request: RETR robots.txt
755 6.128254	2620:101:f000:4901:c5c:0:f:1055	2802:8010:a824:ba00:4e4:6728:54d5:e2a2	FTP	144 Response: 150 Opening BINARY mode data connection for robots.txt (1324 bytes).
819 6.288003	2620:101:f000:4901:c5c:0:f:1055	2802:8010:a824:ba00:4e4:6728:54d5:e2a2	FTP	98 Response: 226 Transfer complete.
5407 36.477861	2802:8010:a824:ba00:4e4:6728:54d5:e2a2	2620:101:f000:4901:c5c:0:f:1055	FTP	82 Request: REST 0
5445 36.632979	2620:101:f000:4901:c5c:0:f:1055	2802:8010:a824:ba00:4e4:6728:54d5:e2a2	FTP	110 Response: 350 Restart position accepted (0).
115. 66.479114	2802:8010:a824:ba00:4e4:6728:54d5:e2a2	2620:101:f000:4901:c5c:0:f:1055	FTP	79 Request: PMD
115. 66.634097	2620:101:f000:4901:c5c:0:f:1055	2802:8010:a824:ba00:4e4:6728:54d5:e2a2	FTP	108 Response: 257 "/" is the current directory
Flags: 0x018 (PSH, ACK) 000. = Reserved: Not set ...0. = Nonce: Not set ...0. = Congestion Window Reduced (CWR): Not set ...0. = ECN-Echo: Not set ...0. = Urgent: Not set ...1. = Acknowledgment: Set ...1. = Push: Set ...0. = Reset: Not set ...0. = Syn: Not set ...0. = Fin: Not set [TCP Flags:AP...] Window: 252 [Calculated window size: 252] [Window size scaling factor: -1 (unknown)] Checksum: 0x2abf [unverified] [Checksum Status: Unverified] Urgent Pointer: 0 > [SEQ/ACK analysis] > [Timestamp]				
0010	b0 9c 00 19 06 40 28 02 80 10 a8 24 ba 00 04 e4	@(. ...\$....		
0020	67 28 54 d5 e2 a2 26 20 01 01 f0 00 49 01 0c 5c	g(T...&I..\		
0030	00 00 0f 10 55 cb 76 00 15 80 fc b9 7f fc 1c	U-vw..		

A continuación se realizó una captura en Wireshark durante 5 minutos sin filtros, donde se observaron diversos protocolos de red, cada uno con funciones específicas:



RTCP: Protocolo complementario de RTP que se utiliza para monitorizar la calidad de transmisión en tiempo real mediante el intercambio de estadísticas entre participantes.

UDP: Protocolo de transporte sin conexión que envía datos sin garantizar entrega, orden ni control de errores, siendo rápido y ligero.

ICMP: Protocolo de control y diagnóstico en redes IP, utilizado para reportar errores y pruebas como el comando ping.

TCP: Protocolo de transporte confiable que establece conexión, garantiza la entrega ordenada de los datos y controla la congestión.

TLSv1.2: Protocolo de seguridad que cifra la comunicación para garantizar confidencialidad, integridad y autenticidad de los datos.

ARP: Protocolo que traduce direcciones IP a direcciones MAC dentro de una red local para permitir la comunicación entre dispositivos.

ICMPv6: Versión de ICMP para IPv6, utilizada para diagnóstico, errores y funciones de descubrimiento en redes IPv6.

STP: Protocolo de capa 2 que evita bucles en redes con switches mediante la creación de una topología libre de ciclos.

QUIC: Protocolo basado en UDP que mejora la velocidad de conexión web, reduciendo la latencia y mejorando la eficiencia frente a TCP.

TLSv1.3: Versión más moderna de TLS que mejora la seguridad y reduce el tiempo de establecimiento de la conexión cifrada.

SSDP: Protocolo de descubrimiento de servicios en redes locales que permite detectar dispositivos automáticamente.

IPv6: Protocolo de capa de red que reemplaza a IPv4, usando direcciones de 128 bits para ampliar el espacio de direccionamiento.

IGMPv3: Protocolo que permite a los hosts gestionar su pertenencia a grupos multicast en redes IP.

DNS: Protocolo que traduce nombres de dominio en direcciones IP para facilitar la navegación en Internet.

FTP: Protocolo de transferencia de archivos entre cliente y servidor que permite subir y descargar archivos en la red.

En conjunto, estos protocolos muestran la interacción de diferentes capas del modelo de red, desde descubrimiento local hasta comunicación segura en Internet.



PUNTO 4 – IP CONFIG / RELEASE / RENEW Y DHCP

Sin filtro ejecutar el comando `ipconfig /release` para liberar la dirección IP de la placa de red y luego ejecutar el comando `ipconfig /renew` para renovar la misma.

Analizar los paquetes DHCP que se presentan.

```
C:\Windows\System32>ipconfig /release

Windows IP Configuration

An error occurred while releasing interface Ethernet : An address has not yet been associated with the network endpoint.

No operation can be performed on WiFi while it has its media disconnected.
No operation can be performed on Local Area Connection* 9 while it has its media disconnected.
No operation can be performed on Local Area Connection* 10 while it has its media disconnected.
No operation can be performed on Bluetooth Network Connection while it has its media disconnected.

C:\Windows\System32>ipconfig /renew

Windows IP Configuration

No operation can be performed on WiFi while it has its media disconnected.
No operation can be performed on Local Area Connection* 9 while it has its media disconnected.
No operation can be performed on Local Area Connection* 10 while it has its media disconnected.
No operation can be performed on Bluetooth Network Connection while it has its media disconnected.

Ethernet adapter Ethernet:

    Connection-specific DNS Suffix  . : 
    IPv6 Address. . . . . : 2802:8010:a824:ba00:ff24:5e99:5026:86a
    Temporary IPv6 Address. . . . . : 2802:8010:a824:ba00:4e4:6728:54d5:e2a2
    Link-local IPv6 Address . . . . . : fe80::7f1f:6602:72bf:f088%13
    IPv4 Address. . . . . : 192.168.1.35
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : fe80::e6ab:89ff:feec:db2%13
                              192.168.1.1

Wireless LAN adapter WiFi:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . : 

Wireless LAN adapter Local Area Connection* 9:

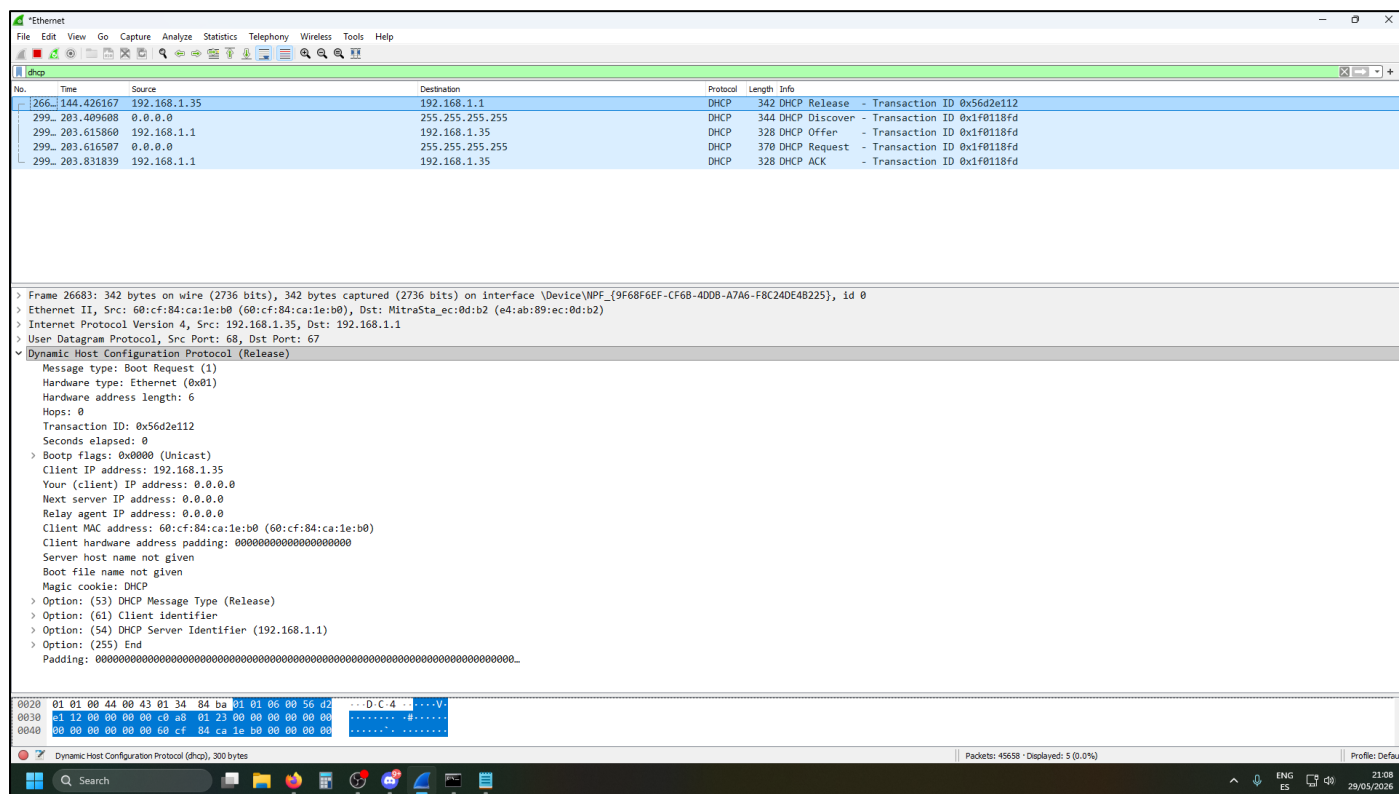
    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . : 

Wireless LAN adapter Local Area Connection* 10:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . : 

Ethernet adapter Bluetooth Network Connection:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . :
```

PUNTO 5 – CUESTIONARIO

A. Para que se utilizan y en qué nivel del modelo OSI se ubican los protocolos PING y TRACE

PING es una herramienta de diagnóstico de red utilizada para verificar si un dispositivo remoto es accesible a través de una red IP y para medir el tiempo que tarda un paquete en ir y volver entre origen y destino (RTT, *Round Trip Time*). Funciona mediante mensajes **ICMP Echo Request** y **ICMP Echo Reply**.

TRACE (o **Traceroute/Tracert**) es una herramienta utilizada para identificar la ruta que siguen los paquetes hasta llegar a un destino. Permite visualizar cada router o salto (*hop*) intermedio y medir el tiempo de respuesta de cada uno, facilitando la detección de problemas de conectividad o congestión en la red.

Tanto **PING** como **TRACE** utilizan el protocolo **ICMP** (Internet Control Message Protocol) y se ubican en la Capa 3 (Red) del modelo OSI, ya que operan directamente sobre el protocolo IP para el envío de mensajes de control y diagnóstico.

B. Para que se utiliza el protocolo ARP, breve descripción del mismo.

El protocolo **ARP (Address Resolution Protocol)** se utiliza para asociar una dirección lógica **IP** con una dirección física **MAC** dentro de una red local.

Cuando un dispositivo necesita enviar datos a otro equipo de la misma red y conoce únicamente su dirección IP, envía una **solicitud ARP (ARP Request)** en forma de difusión (*broadcast*) preguntando qué dispositivo posee esa dirección IP. El equipo correspondiente responde mediante un **ARP Reply**, indicando su dirección MAC. De esta forma, el emisor puede encapsular correctamente los datos en una trama Ethernet y enviarlos al destino.



C. ¿Es posible leer desde el analizador de protocolo el texto descargado del servidor FTP? Mostrar evidencia de la respuesta.

Sí, es posible, siempre que se utilice el protocolo **FTP tradicional** sin mecanismos de cifrado.

FTP transmite la información, incluidos los archivos y las credenciales de acceso, en **texto plano** (*clear text*). Por esta razón, herramientas de análisis de tráfico como **Wireshark** pueden capturar los paquetes y reconstruir el contenido transferido, permitiendo visualizar el texto descargado desde el servidor.

Por ejemplo, se descarga el archivo “robots.txt” del servidor: mirror.csclub.uwaterloo.ca

No.	Time	Source	Destination	Protocol	Length	Info
8011	2026-05-29 21:12:23,832630	129.97.134.71	192.168.1.186	FTP	61	Response: 220-*
8012	2026-05-29 21:12:23,832630	129.97.134.71	192.168.1.186	FTP	128	Response: 220-* Welcome to the University of Waterloo Computer Science Club Mirror
8013	2026-05-29 21:12:23,832630	129.97.134.71	192.168.1.186	FTP	294	Response: 220-*
8014	2026-05-29 21:12:23,832630	129.97.134.71	192.168.1.186	FTP	154	Response: 220-* IPv4 Address: 129.97.134.71
8016	2026-05-29 21:12:23,832812	192.168.1.186	129.97.134.71	FTP	64	Request: AUTH TLS
8040	2026-05-29 21:12:23,998324	129.97.134.71	192.168.1.186	FTP	92	Response: 530 Please login with USER and PASS.
8041	2026-05-29 21:12:23,998583	192.168.1.186	129.97.134.71	FTP	64	Request: AUTH SSL
8073	2026-05-29 21:12:24,165757	129.97.134.71	192.168.1.186	FTP	92	Response: 530 Please login with USER and PASS.
8074	2026-05-29 21:12:24,166981	192.168.1.186	129.97.134.71	FTP	70	Request: USER anonymous
8102	2026-05-29 21:12:24,334148	129.97.134.71	192.168.1.186	FTP	88	Response: 331 Please specify the password.
8103	2026-05-29 21:12:24,334313	192.168.1.186	129.97.134.71	FTP	82	Request: PASS anonymous@example.com
8132	2026-05-29 21:12:24,505138	129.97.134.71	192.168.1.186	FTP	77	Response: 230 Login successful.
8133	2026-05-29 21:12:24,507948	192.168.1.186	129.97.134.71	FTP	61	Request: CWD /
8167	2026-05-29 21:12:24,672086	129.97.134.71	192.168.1.186	FTP	91	Response: 250 Directory successfully changed.
8168	2026-05-29 21:12:24,672264	192.168.1.186	129.97.134.71	FTP	59	Request: PWD
8200	2026-05-29 21:12:24,835932	129.97.134.71	192.168.1.186	FTP	88	Response: 257 "/" is the current directory
8201	2026-05-29 21:12:24,836688	192.168.1.186	129.97.134.71	FTP	62	Request: TYPE A
8246	2026-05-29 21:12:25,002075	129.97.134.71	192.168.1.186	FTP	84	Response: 200 Switching to ASCII mode.
8248	2026-05-29 21:12:25,002255	192.168.1.186	129.97.134.71	FTP	60	Request: PASV
8282	2026-05-29 21:12:25,167964	129.97.134.71	192.168.1.186	FTP	106	Response: 227 Entering Passive Mode (129,97,134,71,254,155).
8283	2026-05-29 21:12:25,168371	192.168.1.186	129.97.134.71	FTP	71	Request: RETR robots.txt
8346	2026-05-29 21:12:25,506661	129.97.134.71	192.168.1.186	FTP	124	Response: 150 Opening BINARY mode data connection for robots.txt (1324 bytes).
8350	2026-05-29 21:12:25,516134	129.97.134.71	192.168.1.186	FTP-DATA	1378	FTP Data: 1324 bytes (PASV) (RETR robots.txt)
8380	2026-05-29 21:12:25,680494	129.97.134.71	192.168.1.186	FTP	78	Response: 226 Transfer complete.
17777	2026-05-29 21:13:41,486340	129.97.134.71	192.168.1.186	FTP	68	Response: 421 Timeout.

> Frame 8350: 1378 bytes on wire (11024 bits), 1378 bytes captured (11024 bits) on interface \Device\NPF_{1339522C-58C5-4658-BC66-C1746148167D}, id 0
 > Ethernet II, Src: 8c:86:dd:23:b6:ae (8c:86:dd:23:b6:ae), Dst: 34:6f:24:29:95:0f (34:6f:24:29:95:0f)
 > Internet Protocol Version 4, Src: 129.97.134.71, Dst: 192.168.1.186
 > Transmission Control Protocol, Src Port: 65179, Dst Port: 50372, Seq: 1, Ack: 1, Len: 1324
 FTP Data (1324 bytes data)
 [Setup frame: 8382]
 [Setup method: PASV]
 [Command: RETR robots.txt]
 Command frame: 8283
 [Current working directory: /]
 > Line-based text data (67 lines)

Se hace click derecho en FTP Data: 1324 bytes (PASV) (RETR robots.exe), en “Seguir” y luego en “Flujo TCP”

The screenshot shows a Wireshark capture of an FTP session. The packet list on the left shows a directory listing (250 Directory successfully changed) and a file transfer (RETR robots.txt). The packet details pane on the right shows the raw data of the file transfer, which is unencrypted text. The text data is shown in hexadecimal and ASCII format, indicating that the data is transmitted in plain text.

Se podrá evidenciar que FTP transmite los datos de un texto plano SIN CIFRADO.





CONCLUSIONES

El desarrollo de este laboratorio nos permitió ver de forma real y directa cómo interactúan los protocolos dentro de nuestra propia red hogareña. Fue muy interesante analizar el "saludo de tres vías" (SYN, SYN-ACK, ACK) al intentar comunicarnos con la página de la universidad y relacionarlo con el contenido que habíamos visto en clase.

En el camino nos topamos con varias dudas y pequeñas complicaciones. Al principio nos trabamos un poco con ARP porque intentamos aplicar filtros por dirección IP en Wireshark, olvidando por un momento que es un protocolo que trabaja en la capa 2 del modelo OSI y requiere un análisis basado en direcciones MAC. También nos generó mucha intriga el misterio del alfabeto cortado (de la "a" a la "w") en el relleno del ping de Windows, lo que nos llevó a investigar a fondo. Además, tuvimos el típico descuido de olvidarnos de sacar la captura de pantalla a tiempo durante la prueba del paquete de 4000 bytes, teniendo que rescatar la imagen directamente de la grabación del video de nuestra resolución.

Como cierre, analizamos mucho, dimos varias vueltas, pero nos gustó usar el analizador de protocolo para ver cómo funciona todo (el chisme) desde adentro. También entendimos que lo que vemos en la pantalla a veces puede engañarnos, ya que la diferencia de tamaños en las consultas ARP nos enseñó que Wireshark intercepta nuestros propios paquetes antes de que la tarjeta de red física les inyecte el relleno obligatorio para cumplir con el estándar de Ethernet.